



الجامعة اللبنانية
UNIVERSITE LIBANAISE

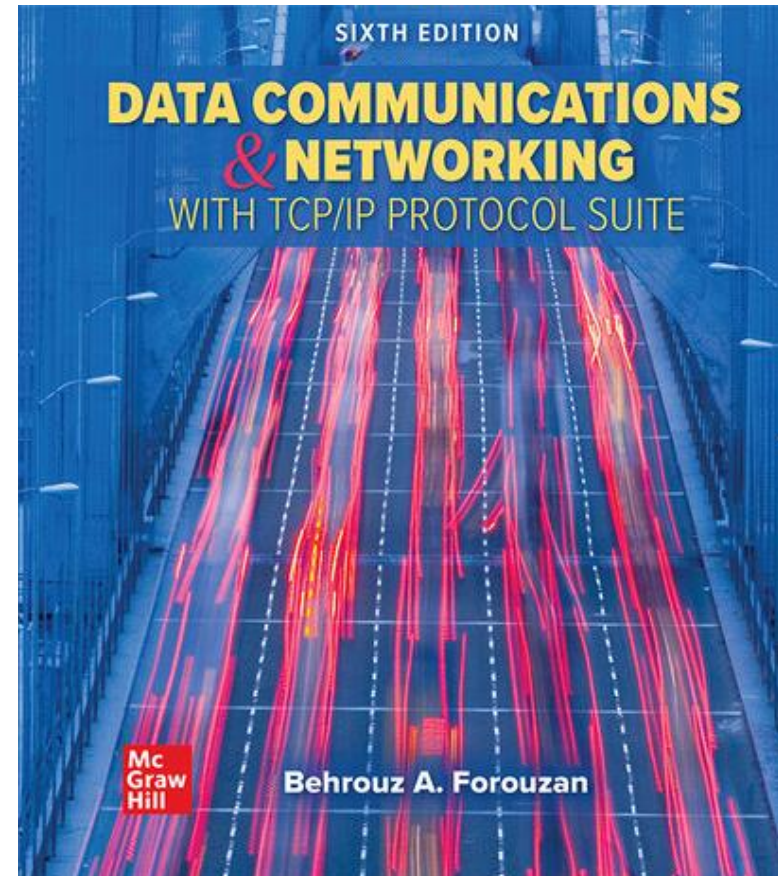
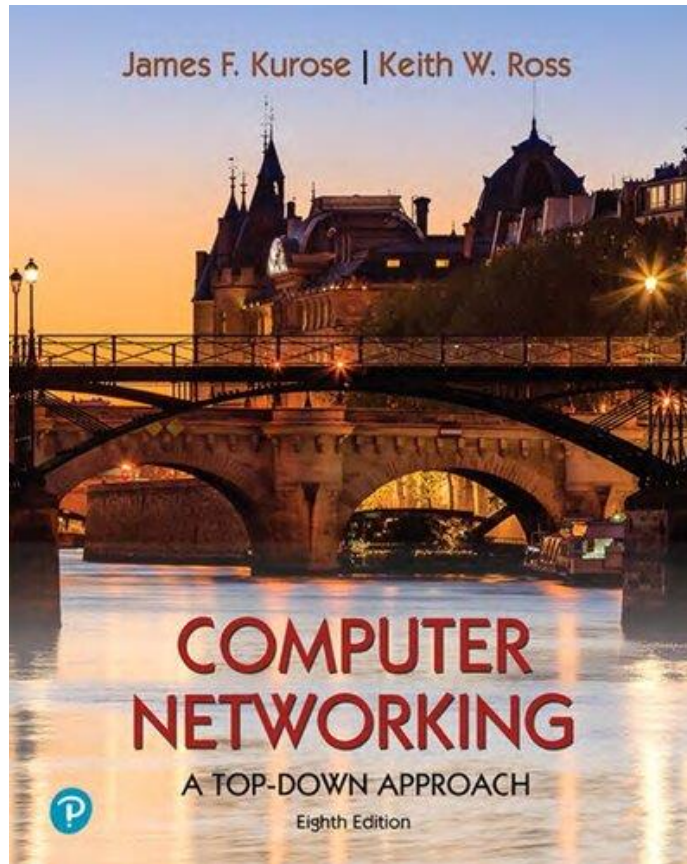


I3304

Network administration and security

Ahmad Fadlallah

Reference Textbooks





Outline

- Introduction
 - ⊙ Introduction to the course
 - ⊙ Recall Network Basics (I2208)
- Network Layer
 - ⊙ Static Routing
 - ⊙ Dynamic Routing
 - Dynamic Routing Algorithm
 - Dynamic Routing Protocols
 - ⊙ NAT (Network Address Translation)
 - ⊙ IPv6
- Transport Layer
 - ⊙ Function of the transport layer
 - ⊙ UDP Protocol
 - ⊙ TCP Protocol
 - Connection management
 - Flow control
 - Congestion control
- Application Layer
 - HTTP protocol
 - FTP protocol
 - Telnet
 - Mail protocols
 - DNS
 - P2P Applications
- Introduction to Security
 - Security services
 - Cryptography
 - Digital Signature
 - Principle of network security protocols

References



- The slides are based on the:
 - ◉ Jim Kurose, Keith Ross Slides for the Computer Networking: A Top-Down Approach, 8th edition, Pearson, 2020
 - ◉ Cisco Networking Academy Program, Routing and Switching Essentials v6.0



Application layer: Outline

- Introduction
- Web and HTTP
- File Transfer Protocol (FTP)
- Telnet
- Mail protocols
- Domain Name System (DNS)
- Peer-to-Peer (P2P) Applications



Introduction

Web and HTTP

File Transfer Protocol (FTP)

Telnet

Mail protocols

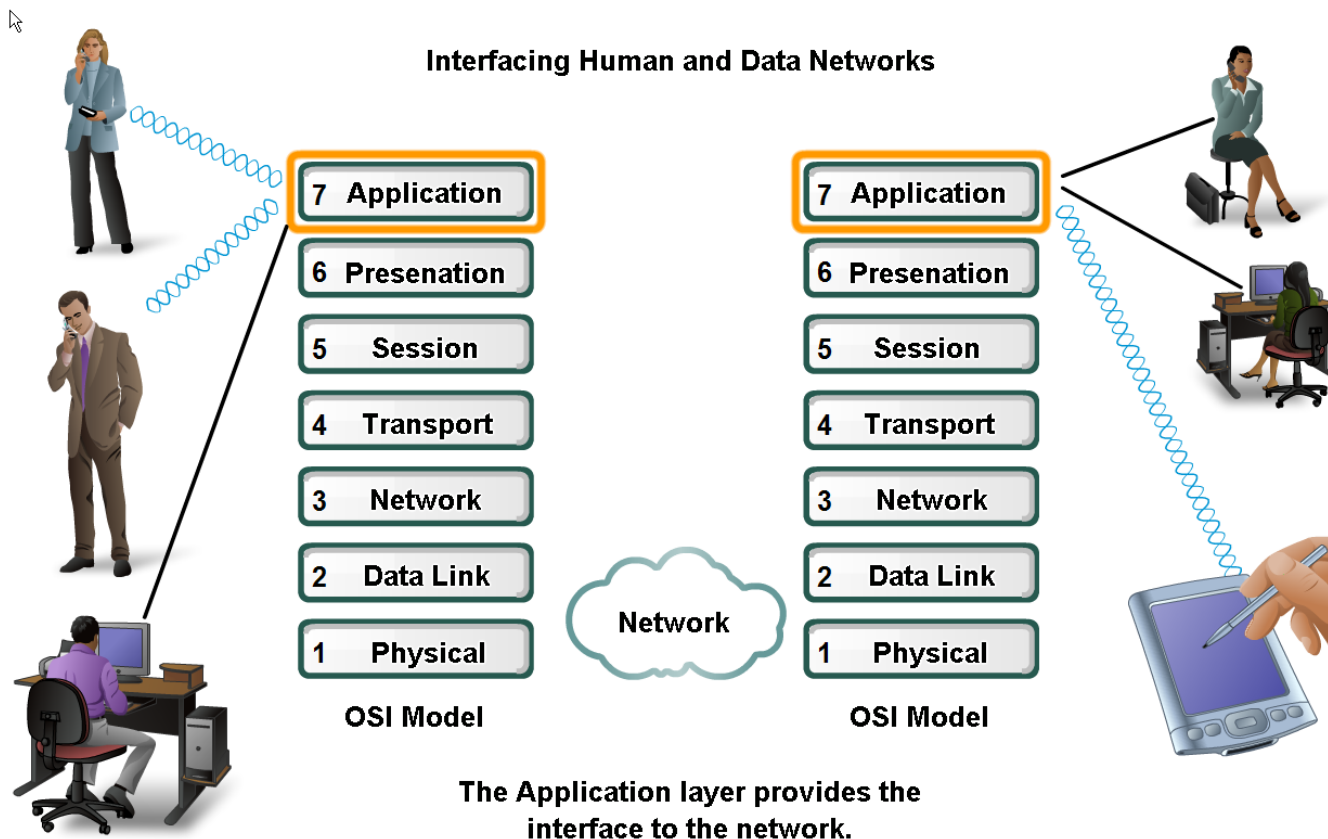
Domain Name System (DNS)

Peer-to-Peer (P2P) Applications

Introduction



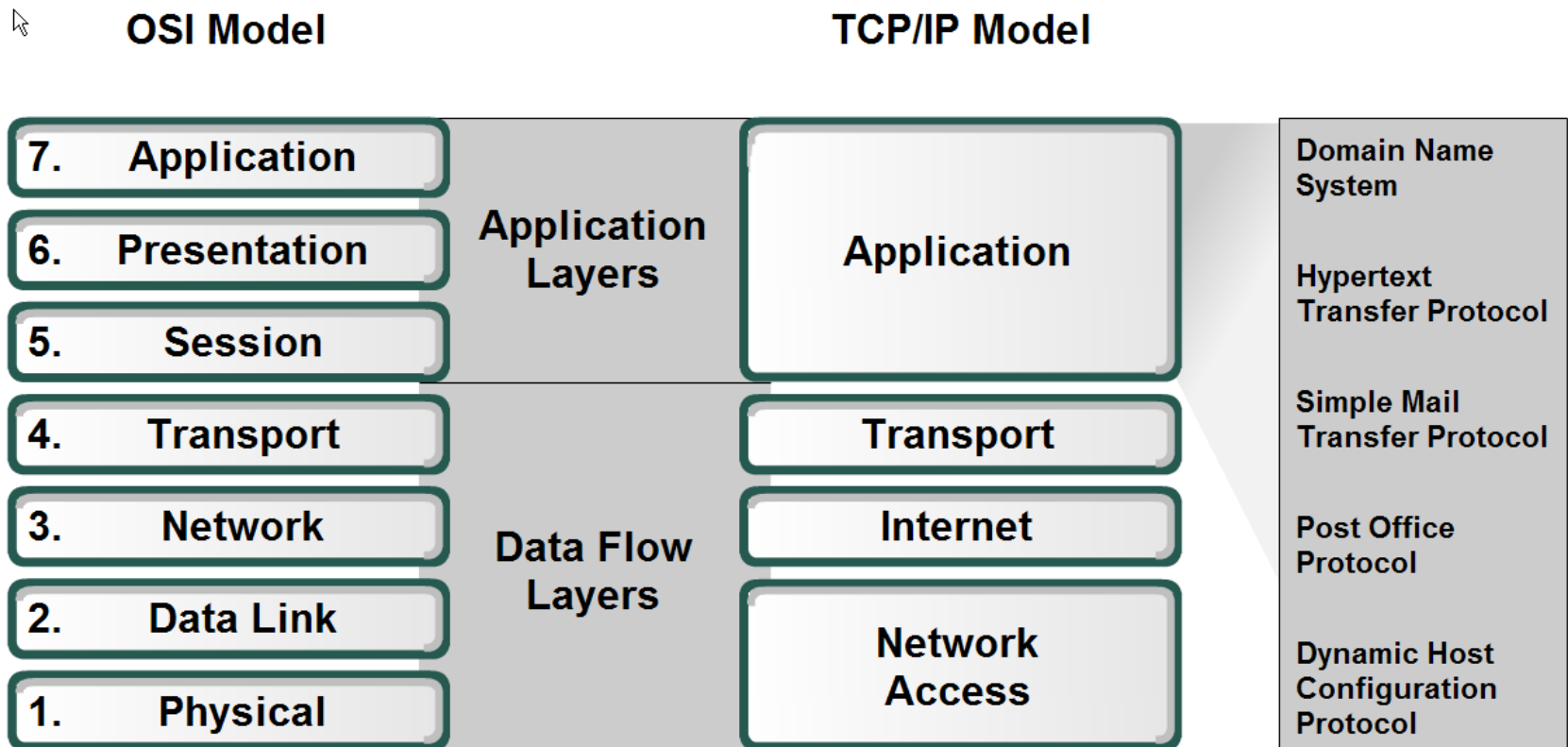
- Application Layer – allows user to interface with the network!



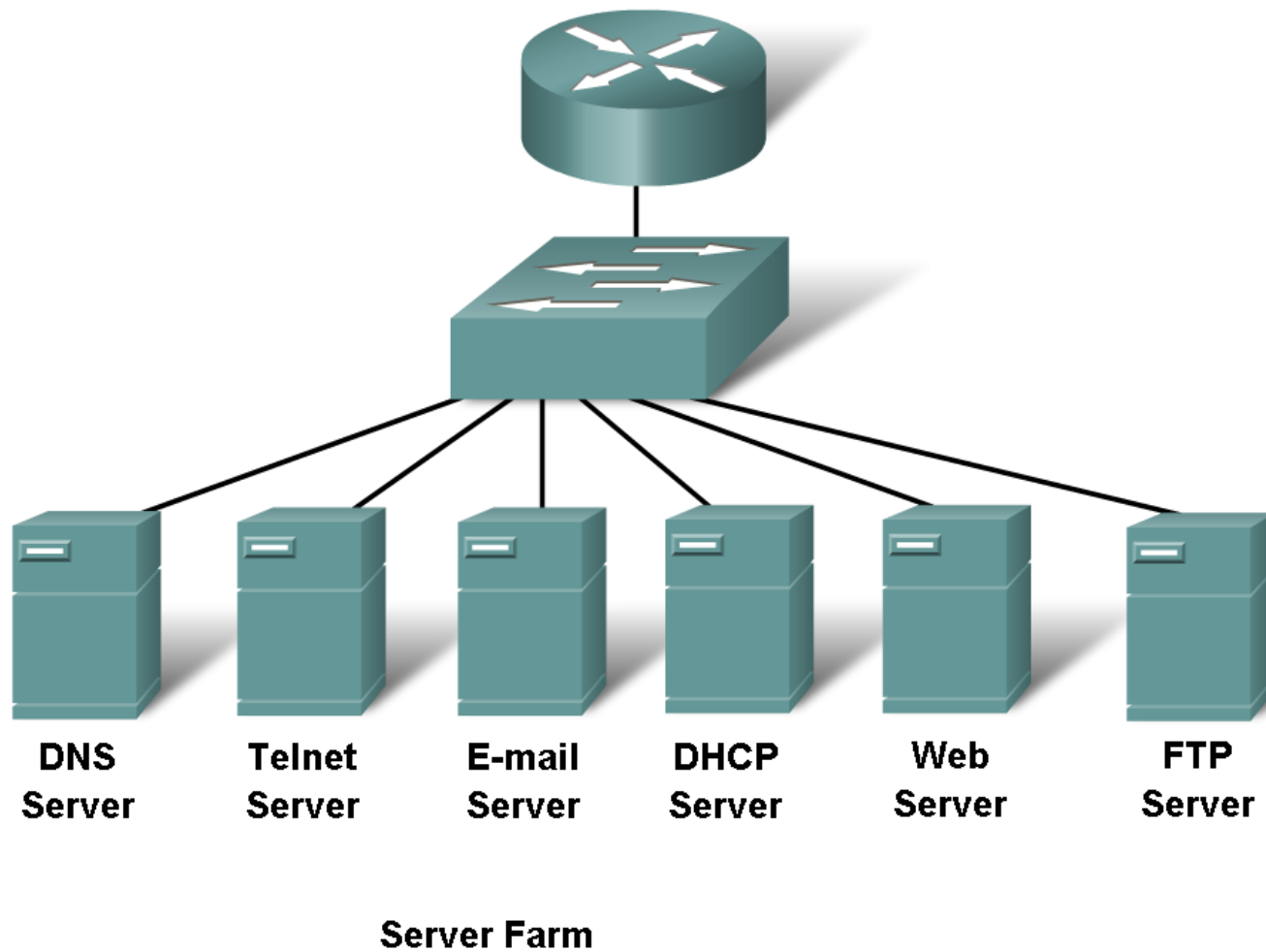


OSI model vs. TCP/IP Model

- Application Layer – Provides the interface between the applications on either end of the network.



Protocols and networks



Protocols



DNS	Matches domain names with IP addresses
HTTP	Used to transfer data between clients/servers using a web browser
SMTP & POP3	Used to send email messages from clients to servers over the internet
FTP	allows the download/upload of files between a client/server
Telnet	allows users to login to a host from a remote location and take control as if they were sitting at the machine (virtual connection)
DHCP	Assigns IP addresses, subnet masks, default gateways, DNS servers, etc.. To users as they login the network



Common Port Numbers

- TCP

- ⦿ FTP – 20-21

- ⦿ Telnet – 23

- ⦿ SMTP – 25

- ⦿ DNS – 53 (Both TCP & UDP)

- ⦿ HTTP – 80

- UDP

- ⦿ DHCP – 67 & 68

- ⦿ POP – 110

Application Layer Software



- **Applications**

- ⊙ Provide the human (user) interface.
- ⊙ Relies on lower layers to complete the communication process.

- **Services**

- ⊙ Establish an interface to the network where protocols provide the rules and formats that govern how data is treated..

Application architectures



- Client-server
- Peer-to-peer (P2P)
- Hybrid of client-server and P2P

Client-Server Paradigm

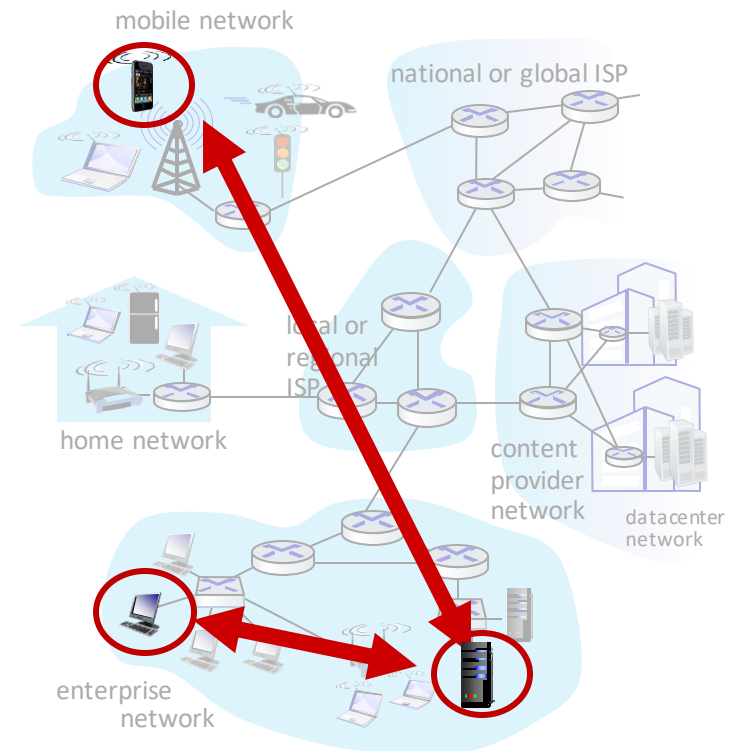


- **Server:**

- ⦿ Always-on host
- ⦿ Permanent IP address
- ⦿ Often in data centers, for scaling

- **Clients:**

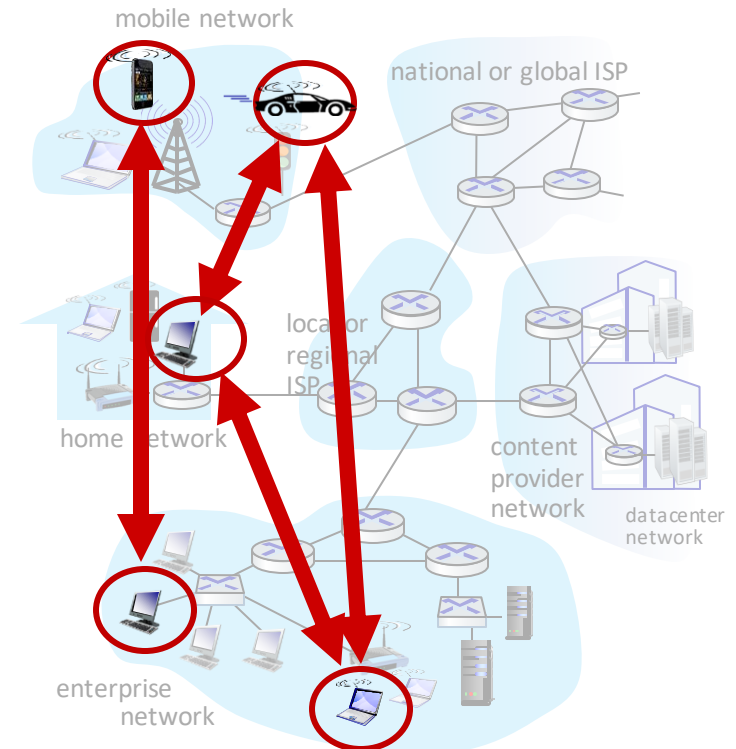
- ⦿ Contact, communicate with server
- ⦿ May be intermittently connected
- ⦿ May have dynamic IP addresses
- ⦿ Do not communicate directly with each other
- ⦿ Examples: HTTP, IMAP, FTP





Peer-to-Peer (P2P) architecture

- No always-on server
- Arbitrary end systems directly communicate
- Peers request service from other peers, provide service in return to other peers
 - ◉ *Self scalability* – new peers bring new service capacity, as well as new service demands
- Peers are intermittently connected and change IP addresses
 - ◉ Complex management
- Example: P2P file sharing





Hybrid of client-server and P2P

- **Skype**

- ⊙ Voice-over-IP P2P application
- ⊙ Centralized server: finding address of remote party
- ⊙ Client-client connection: direct (not through server)

- **Instant messaging**

- ⊙ Chatting between two users is P2P
- ⊙ Centralized service: client presence detection/location
 - User registers its IP address with central server when it comes online
 - User contacts central server to find IP addresses of buddies



Processes communicating

process: program running within a host

- within same host, two processes communicate using **Inter-Process Communication** (defined by OS)
- Processes in different hosts communicate by exchanging **messages**

Clients, Servers

client process: process that initiates communication

server process: process that waits to be contacted

- note: applications with P2P architectures have client processes & server processes



Application-Layer Protocol defines

- **Types of messages exchanged**

- ⊙ e.g., Request, Response

- **Message syntax:**

- ⊙ What fields in messages & how fields are delineated

- **Message semantics**

- ⊙ Meaning of information in fields

- **Rules** for when and how processes send & respond to messages

Public-domain protocols:

- defined in RFCs
- allows for interoperability
- e.g., HTTP, SMTP

Proprietary protocols:

- e.g., Skype, Zoom

What transport service does an app need?



- **Reliability**

- ⊙ Some apps (e.g., audio) can tolerate some loss
- ⊙ Other apps (e.g., file transfer, telnet) require 100% reliable data transfer

- **Timing**

- ⊙ Some apps (e.g., Internet telephony, interactive games) require low delay to be “effective”

- **Throughput**

- ⊙ Some apps (e.g., multimedia) require minimum amount of throughput to be “effective”
- ⊙ Other apps (“elastic apps”) make use of whatever throughput they get

- **Security**

- ⊙ Encryption, data integrity, ...



Transport service requirements: common apps

Application	Data loss	Throughput	Time sensitive
File transfer/ Download	No loss	Elastic	No
E-mail	No loss	Elastic	No
Web documents	No loss	Elastic	No
Real-time audio/video	Loss-tolerant	Audio: 5Kbps-1Mbps Video:10Kbps-5Mbps	Yes, 10's msec
streaming audio/video	Loss-tolerant	Same as above	Yes, few secs
Interactive games	Loss-tolerant	Kbps+	Yes, 10's msec
Text Messaging	No loss	Elastic	Yes and no



Internet transport protocols services

TCP service:

- *connection-oriented*: setup required between client and server processes
- *reliable transport* between sending and receiving process
- *flow control*: sender won't overwhelm receiver
- *congestion control*: throttle sender when network overloaded
- *does not provide*: timing, minimum throughput guarantees, security

UDP service:

- unreliable data transfer between sending and receiving process
- does not provide: connection setup, reliability, flow control, congestion control, timing, throughput guarantee, or security

Q: why bother? Why is there a UDP?

Internet applications, and transport protocols



Application	Application Layer Protocol	Transport Protocol
File Transfer/Download	FTP [RFC 959]	TCP
E-Mail	SMTP [RFC 5321]	TCP
Web Documents	HTTP 1.1 [RFC 7320]	TCP
Internet Telephony	SIP [RFC 3261], RTP [RFC 3550], Or Proprietary	TCP or UDP
Streaming Audio/Video	HTTP [RFC 7320], DASH	TCP
Interactive Games	WOW, FPS (Proprietary)	UDP or TCP



Introduction

Web and HTTP

File Transfer Protocol (FTP)

Telnet

Mail protocols

Domain Name System (DNS)

Peer-to-Peer (P2P) Applications



Web and HTTP

First, a quick review...

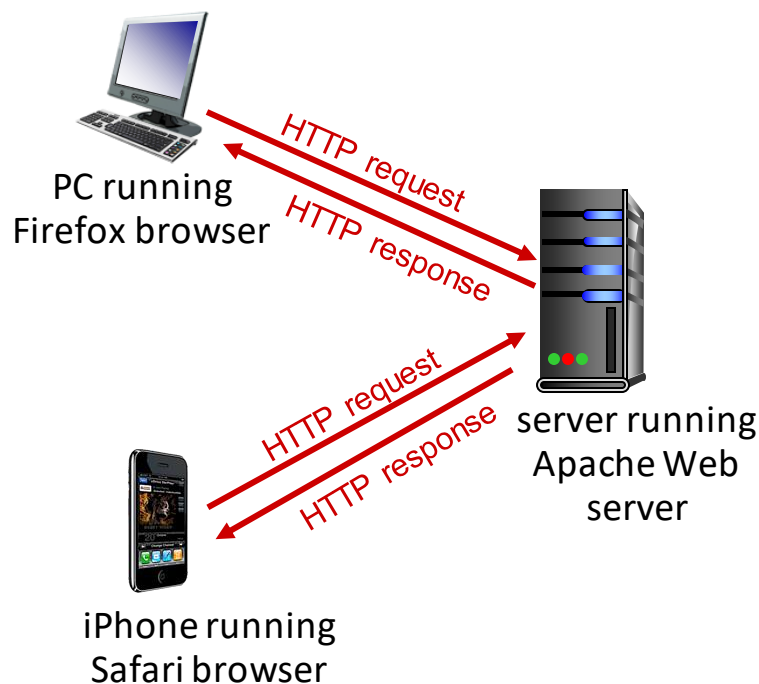
- web page consists of *objects*, each of which can be stored on different Web servers
- object can be HTML file, JPEG image, Java applet, audio file,...
- web page consists of *base HTML-file* which includes *several referenced objects, each* addressable by a *URL*, e.g.,

www.someschool.edu / someDept/pic.gif
host name path name



HTTP overview

- HTTP: HyperText Transfer Protocol
- Web's application-layer protocol
- **Client/Server model:**
 - ⊙ **Client:** browser that requests, receives, (using HTTP protocol) and “displays” Web objects
 - ⊙ **Server:** Web server sends (using HTTP protocol) objects in response to requests





HTTP overview (continued)

HTTP uses TCP (1.0, 1.1, 2.0):

- *Client* initiates TCP connection (creates socket) to server, port 80
- *Server* accepts TCP connection from client
- HTTP messages (application-layer protocol messages) exchanged between browser (HTTP client) and Web server (HTTP server)
- TCP connection closed

HTTP is “stateless”

- Server maintains no information about past client requests

aside
protocols that maintain “state” are complex!

- Past history (state) must be maintained
- if server/client crashes, their views of “state” may be **inconsistent**, must be reconciled



HTTP connections: two types

Non-persistent HTTP

1. TCP connection opened
- 2. at most one object sent** over TCP connection
3. TCP connection closed

**downloading multiple objects
required multiple connections**

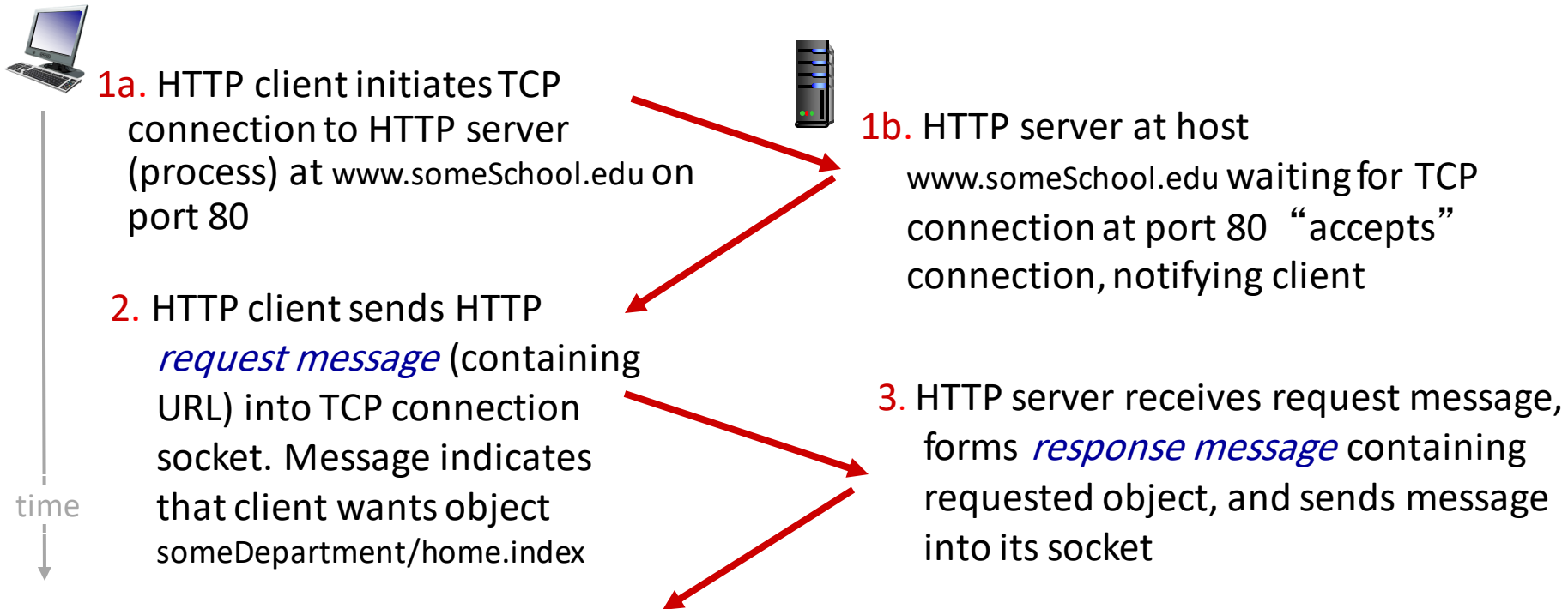
Persistent HTTP

1. TCP connection opened to a server
- 2. multiple objects can be sent** over *single* TCP connection between client, and that server
3. TCP connection closed

Non-persistent HTTP: example



User enters URL: `www.someSchool.edu/someDepartment/home.index`
(containing text, references to 10 jpeg images)





Non-persistent HTTP: example (cont.)

User enters URL: `www.someSchool.edu/someDepartment/home.index`
(containing text, references to 10 jpeg images)



5. HTTP client receives response message containing html file, displays html. Parsing html file, finds 10 referenced jpeg objects

6. Steps 1-5 repeated for each of 10 jpeg objects

4. HTTP server closes TCP connection.



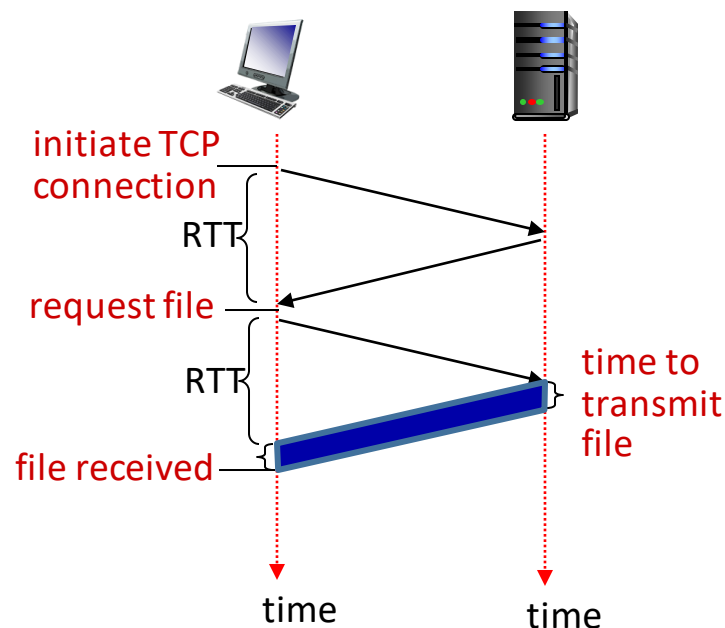


Non-persistent HTTP: response time

RTT (definition): time for a small packet to travel from client to server and back

HTTP response time (per object):

- one RTT to initiate TCP connection
- one RTT for HTTP request and first few bytes of HTTP response to return
- object/file transmission time



Non-persistent HTTP response time = $2RTT + \text{file transmission time}$



Persistent HTTP (HTTP 1.1)

Non-persistent HTTP issues:

- requires 2 RTTs per object
- OS overhead for *each* TCP connection
- browsers often open multiple parallel TCP connections to fetch referenced objects in parallel

Persistent HTTP (HTTP1.1):

- server leaves connection open after sending response
- subsequent HTTP messages between same client/server sent over open connection
- client sends requests as soon as it encounters a referenced object
- as little as one RTT for all the referenced objects (cutting response time in half)

HTTP request message



- two types of HTTP messages: *request, response*
- **HTTP request message:**
 - ASCII (human-readable format)

request line (GET, POST, HEAD commands) →

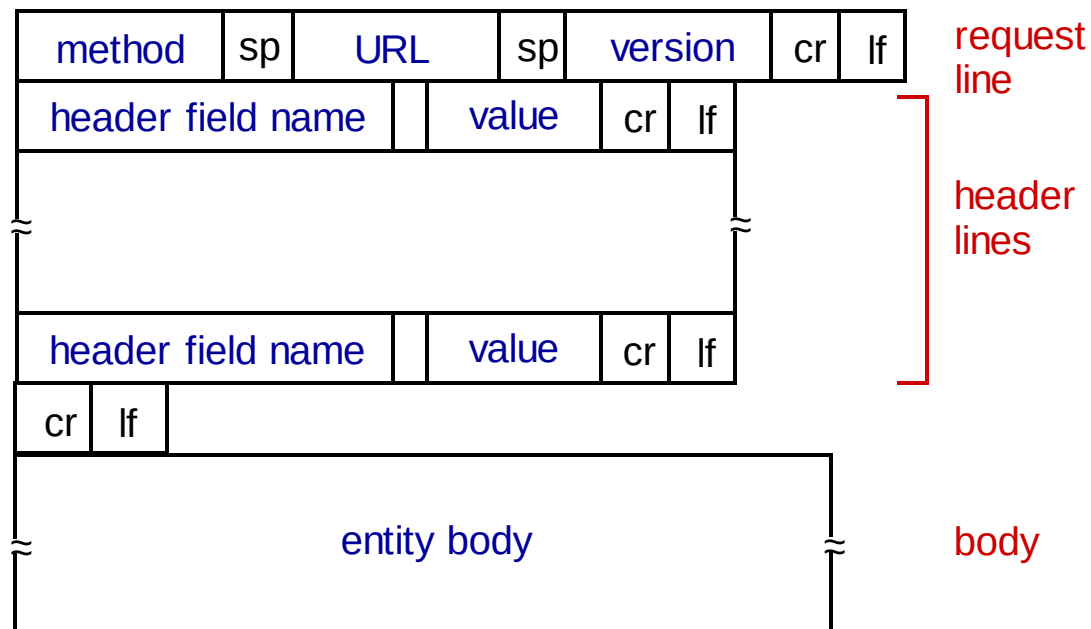
header lines {

```
GET /index.html HTTP/1.1\r\n
Host: www-net.cs.umass.edu\r\n
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X
10.15; rv:80.0) Gecko/20100101 Firefox/80.0 \r\n
Accept: text/html,application/xhtml+xml\r\n
Accept-Language: en-us,en;q=0.5\r\n
Accept-Encoding: gzip,deflate\r\n
Connection: keep-alive\r\n
\r\n
```

carriage return, line feed at start of line indicates end of header lines →

carriage return character
line-feed character

HTTP request message: general format





Other HTTP request messages

- **POST method:**

- ⊙ Web page often includes form input
- ⊙ User input sent from client to server **in entity body** of HTTP POST request message

- **GET method (for sending data to server):**

- ⊙ Include **user data in URL field** of HTTP GET request message (following a '?'):

`www.somesite.com/animalsearch?ant`

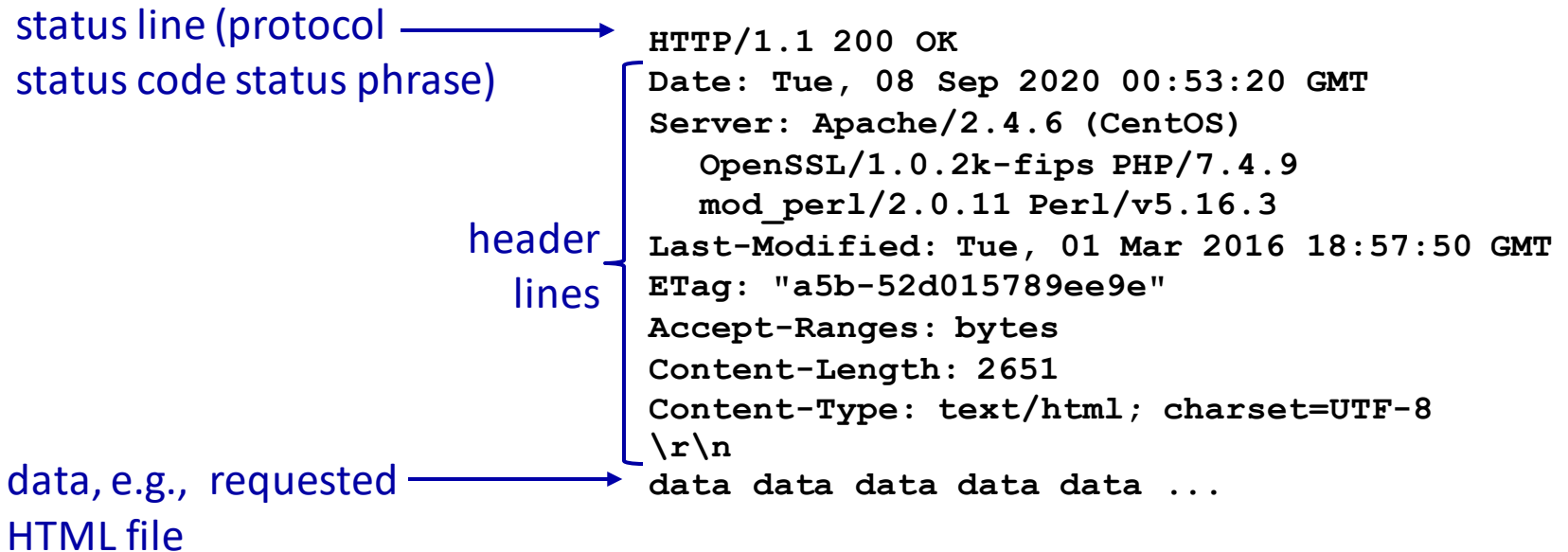
- **HEAD method:**

- ⊙ Requests headers (only) that would be returned if specified URL were requested with an HTTP GET method.

- **PUT method:**

- ⊙ Uploads new file (object) to server
- ⊙ Completely replaces file that exists at specified URL with content in entity body of POST HTTP request message

HTTP response message





HTTP response status codes

- Status code appears in 1st line in server-to-client response message.

- Some sample codes:

⦿ 200 OK

- request succeeded, requested object later in this message

⦿ 301 Moved Permanently

- requested object moved, new location specified later in this message (in Location: field)

⦿ 400 Bad Request

- request msg not understood by server

⦿ 404 Not Found

- requested document not found on this server

⦿ 505 HTTP Version Not Supported



Maintaining user/server state: cookies

Recall: HTTP GET/response interaction is *stateless*

- No notion of multi-step exchanges of HTTP messages to complete a Web “transaction”
 - ⊙ No need for client/server to track “state” of multi-step exchange
 - ⊙ All HTTP requests are independent of each other
 - ⊙ No need for client/server to “recover” from a partially-completed-but-never-completely-completed transaction



Maintaining user/server state: cookies

Web sites and client browser use *cookies* to maintain some state between transactions

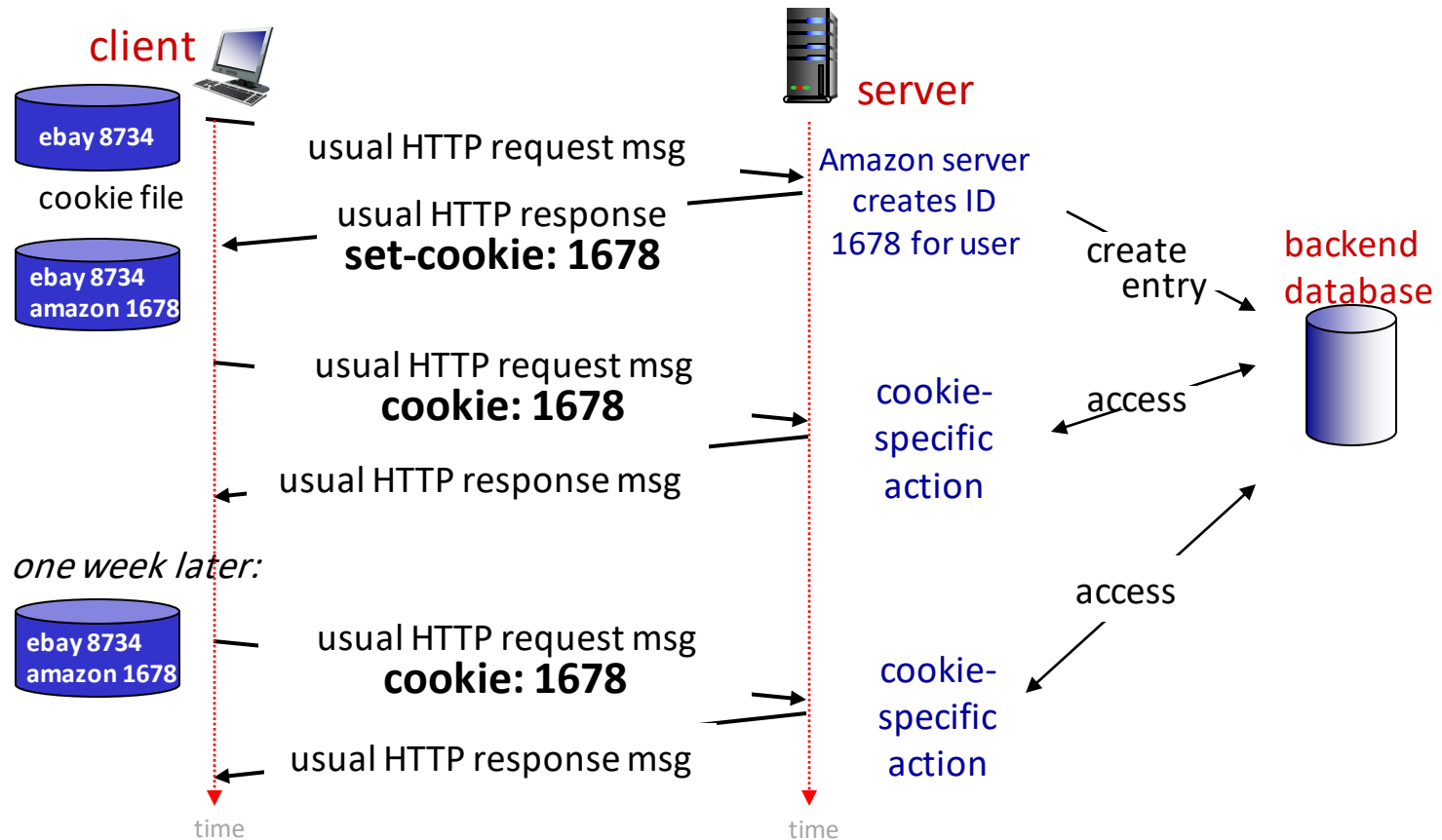
four components:

1. *cookie* header line of *HTTP response* message
2. *cookie* header line in next *HTTP request* message
3. *cookie file* kept on user's host, managed by user's browser
4. back-end database at Web site

Example:

- Susan uses browser on laptop, visits specific e-commerce site for first time
- when initial HTTP requests arrives at site, site creates:
 - unique ID (aka “cookie”)
 - entry in backend database for ID
- subsequent HTTP requests from Susan to this site will contain cookie ID value, allowing site to “identify” Susan

Maintaining user/server state: cookies





HTTP cookies: comments

What cookies can be used for:

- Authorization
- Shopping carts
- Recommendations
- User session state (Web e-mail)

Challenge: How to keep state?

- *at protocol endpoints:* maintain state at sender/receiver over multiple transactions
- *in messages:* cookies in HTTP messages carry state

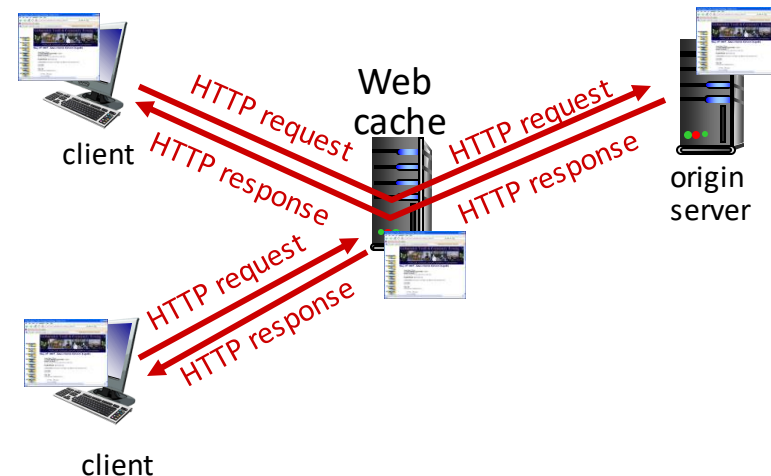
aside cookies and privacy:

- Cookies permit sites to *learn* a lot about you on their site.
- Third party persistent cookies (tracking cookies) allow common identity (cookie value) to be tracked across multiple web sites



Web caches

- **Goal**: satisfy client requests without involving origin server
- User configures browser to point to a (local) Web cache
- Browser sends all HTTP requests to cache
 - ◉ if object in cache: cache returns object to client
 - ◉ else cache requests object from origin server, caches received object, then returns object to client





Web caches (aka proxy servers)

- Web cache acts as both client and server
 - ⊙ Server for original requesting client
 - ⊙ Client to origin server
- Server tells cache about object's allowable caching in response header:

```
Cache-Control: max-age=<seconds>
```

```
Cache-Control: no-cache
```

- **Why Web caching?**

- ⊙ Reduce response time for client request
 - cache is closer to client
- ⊙ Reduce traffic on an institution's access link
- ⊙ Internet is dense with caches
 - enables “poor” content providers to more effectively deliver content

Caching example



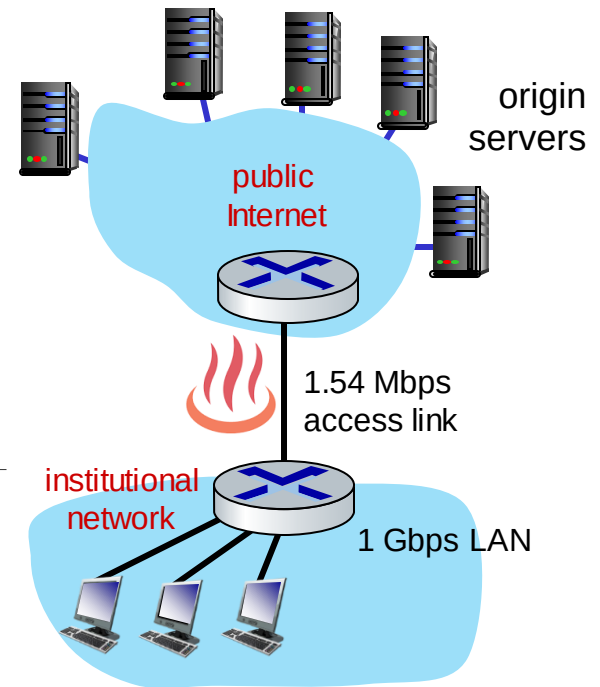
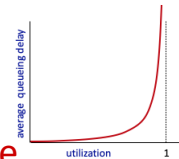
Scenario:

- access link rate: 1.54 Mbps
- RTT from institutional router to server: 2 sec
- web object size: 100K bits
- average request rate from browsers to origin servers: 15/sec
 - avg data rate to browsers: 1.50 Mbps

Performance:

- access link utilization = .97
- LAN utilization: .0015
- end-end delay = Internet delay +
access link delay + LAN delay
= 2 sec + minutes + usecs

problem: large
queueing delays at high
utilization!



Option 1: buy a faster access link



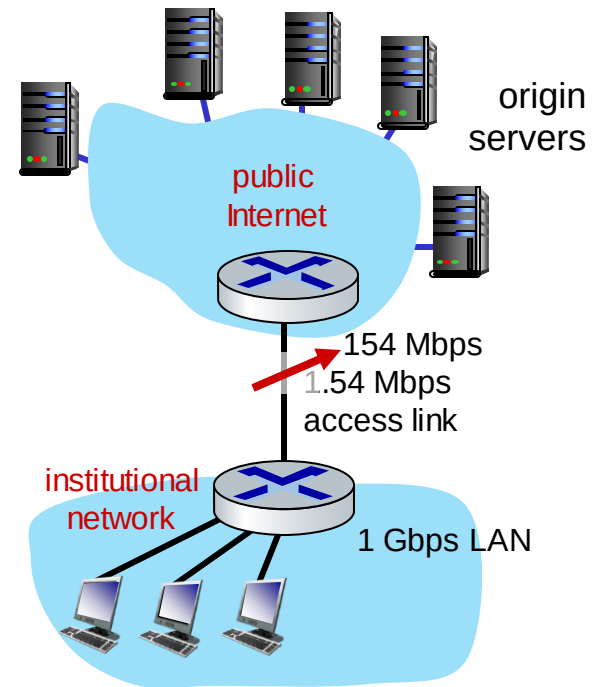
Scenario:

- access link rate: ~~1.54~~ 154 Mbps
- RTT from institutional router to server: 2 sec
- web object size: 100K bits
- average request rate from browsers to origin servers: 15/sec
 - avg data rate to browsers: 1.50 Mbps

Performance:

- access link utilization = ~~.97~~ .0097
- LAN utilization: .0015
- end-end delay = Internet delay +
access link delay + LAN delay
= 2 sec + ~~minutes~~ usecs

Cost: faster access link (expensive!) → msec



Option 2: install a web cache



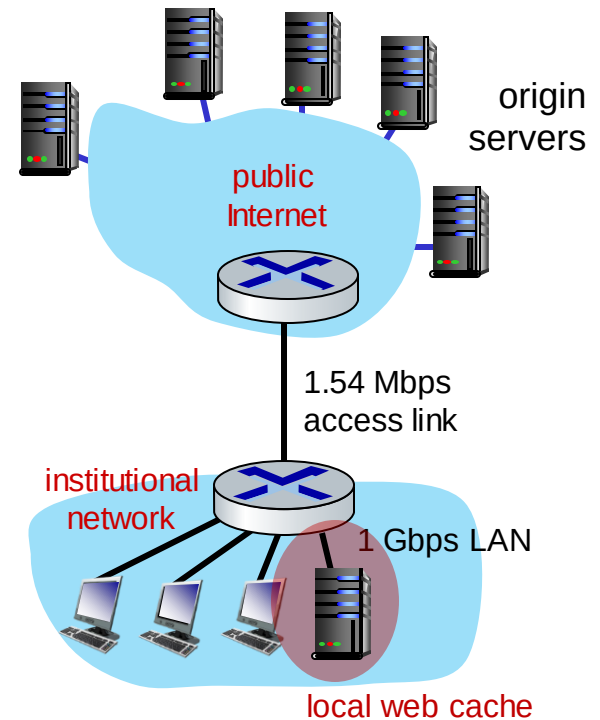
Scenario:

- access link rate: 1.54 Mbps
- RTT from institutional router to server: 2 sec
- web object size: 100K bits
- average request rate from browsers to origin servers: 15/sec
 - avg data rate to browsers: 1.50 Mbps

Cost: web cache (cheap!)

Performance:

- LAN utilization: .?
 - access link utilization = ?
 - average end-end delay = ?
- How to compute link utilization, delay?*

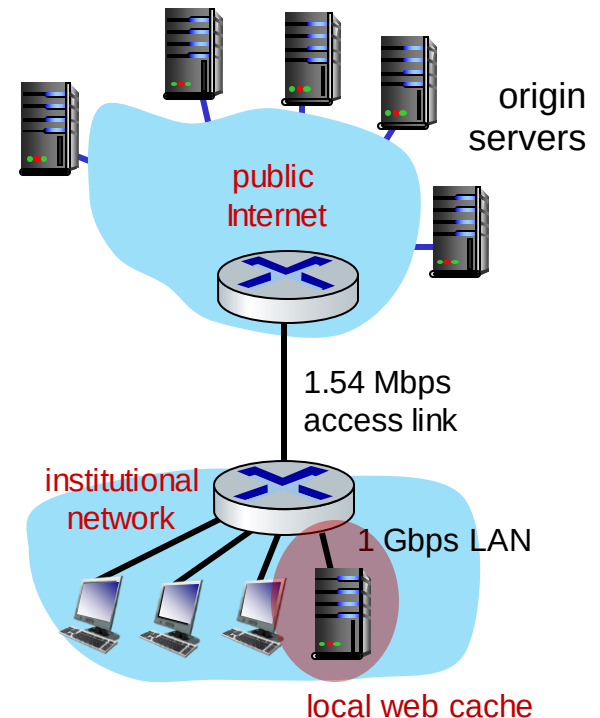


Calculating access link utilization, end-end delay with cache:



suppose cache hit rate is 0.4:

- 40% requests served by cache, with low (msec) delay
- 60% requests satisfied at origin
 - rate to browsers over access link
 $= 0.6 * 1.50 \text{ Mbps} = .9 \text{ Mbps}$
 - access link utilization $= 0.9 / 1.54 = .58$ means low (msec) queueing delay at access link
- average end-end delay:
 $= 0.6 * (\text{delay from origin servers})$
 $+ 0.4 * (\text{delay when satisfied at cache})$
 $= 0.6 (2.01) + 0.4 (\sim \text{msecs}) = \sim 1.2 \text{ secs}$



lower average end-end delay than with 154 Mbps link (and cheaper too!)

Conditional GET



Goal: don't send object if cache has up-to-date cached version

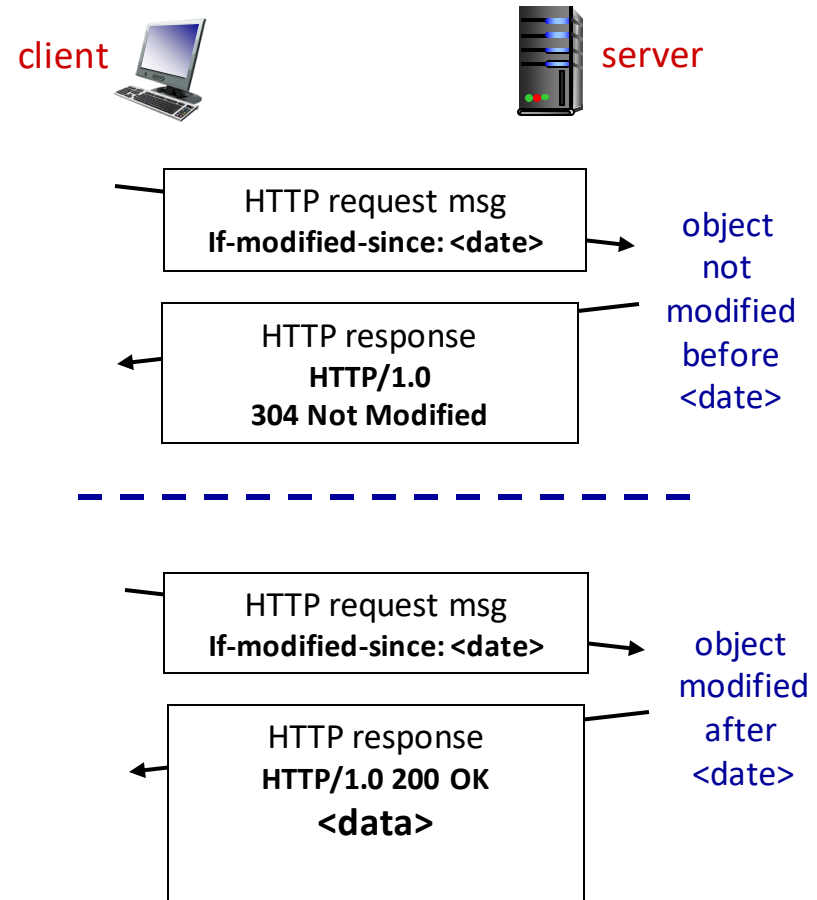
- no object transmission delay (or use of network resources)

- **client:** specify date of cached copy in HTTP request

If-modified-since: <date>

- **server:** response contains no object if cached copy is up-to-date:

HTTP/1.0 304 Not Modified



HTTP/2



- **Key goal:** decreased delay in multi-object HTTP requests
- *HTTP1.1*: introduced **multiple, pipelined GETs over single TCP connection**
 - ⊙ Server responds in-order (FCFS: first-come-first-served scheduling) to GET requests
 - ⊙ with FCFS, small object may have to wait for transmission (**head-of-line (HOL) blocking**) behind large object(s)
 - ⊙ Loss recovery (retransmitting lost TCP segments) stalls object transmission

HTTP/2

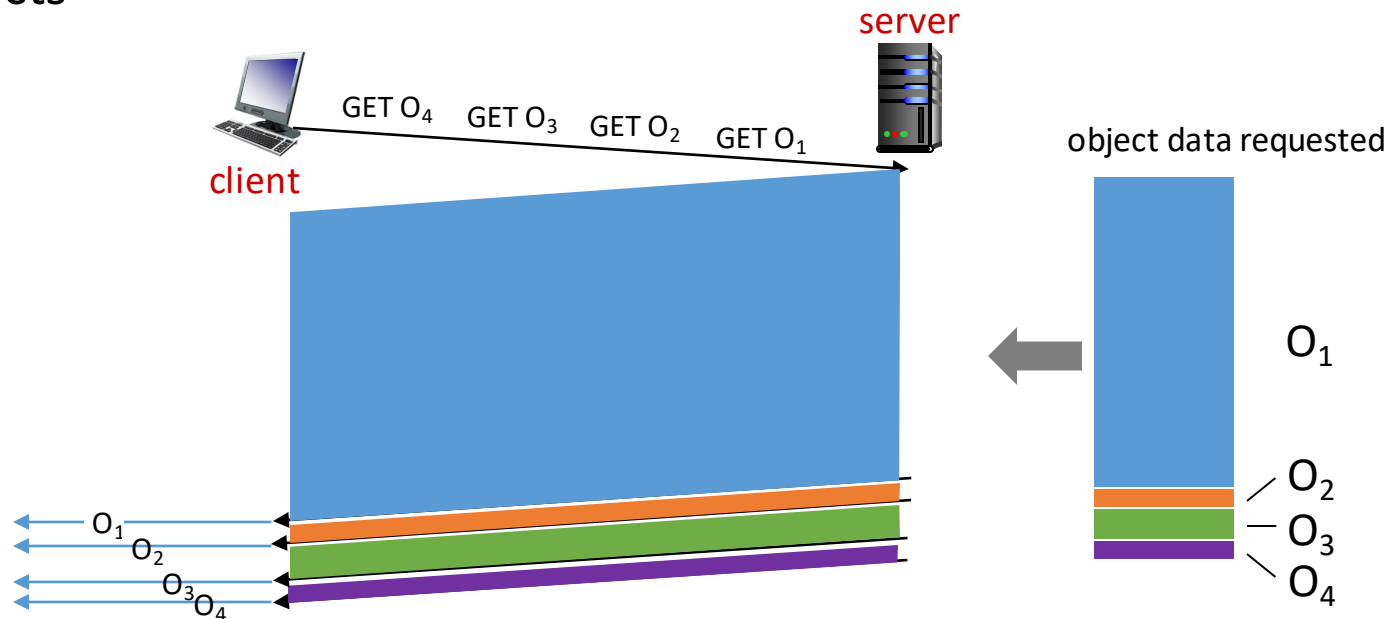


- **Key goal:** decreased delay in multi-object HTTP requests
- *HTTP/2*: [RFC 7540, 2015] increased flexibility at server in sending objects to client:
 - ⊙ Methods, Status Codes, most header fields unchanged from HTTP 1.1
 - ⊙ Transmission order of requested objects based on client-specified object priority (not necessarily FCFS)
 - ⊙ Push unrequested objects to client
 - ⊙ Divide objects into frames, schedule frames to mitigate HOL blocking

HTTP/2: mitigating HOL blocking



HTTP 1.1: client requests 1 large object (e.g., video file) and 3 smaller objects

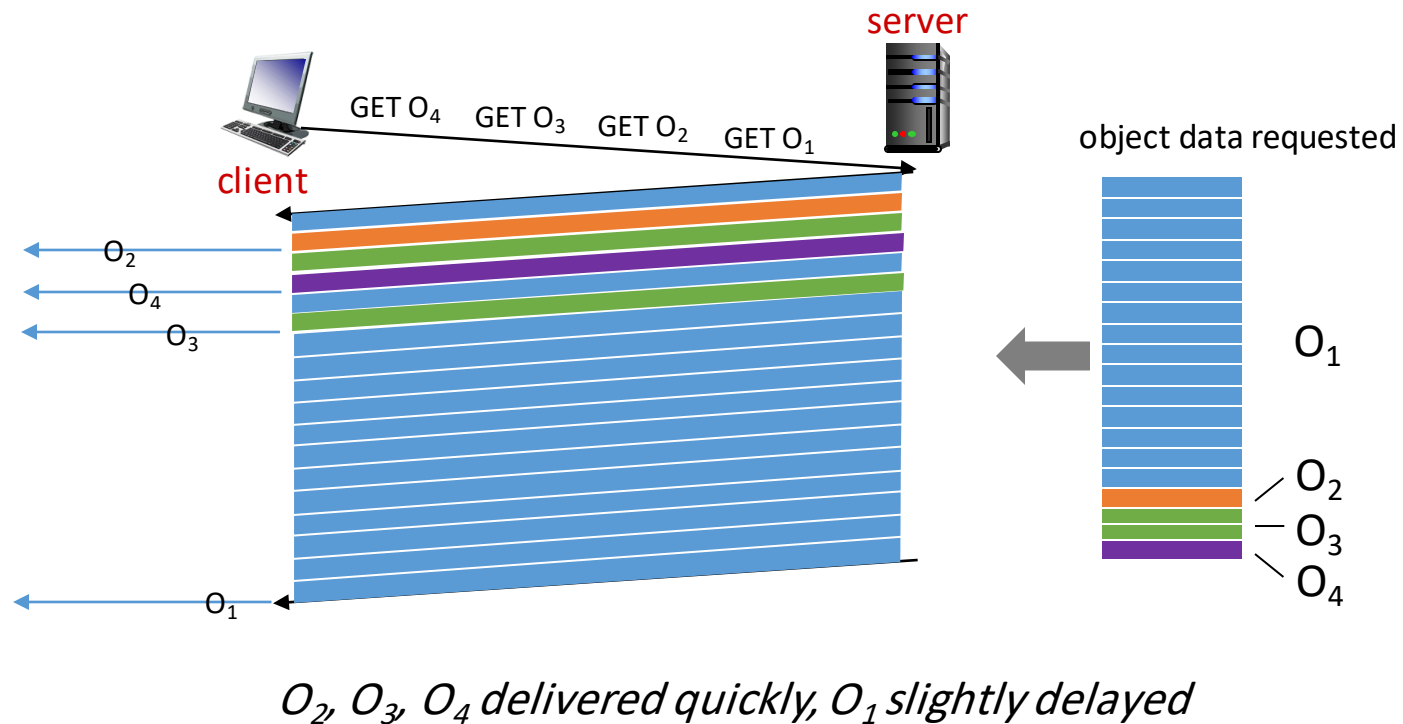


objects delivered in order requested: O_2 , O_3 , O_4 wait behind O_1

HTTP/2: mitigating HOL blocking



HTTP/2: objects divided into frames, frame transmission interleaved





HTTP/2 to HTTP/3

HTTP/2 over single TCP connection means:

- Recovery from packet loss still stalls all object transmissions
 - ⊙ as in HTTP 1.1, browsers have incentive to open multiple parallel TCP connections to reduce stalling, increase overall throughput
- No security over vanilla TCP connection
- **HTTP/3**: adds security, per object error- and congestion-control (more pipelining) over UDP



Introduction

Web and HTTP

File Transfer Protocol (FTP)

Telnet

Mail protocols

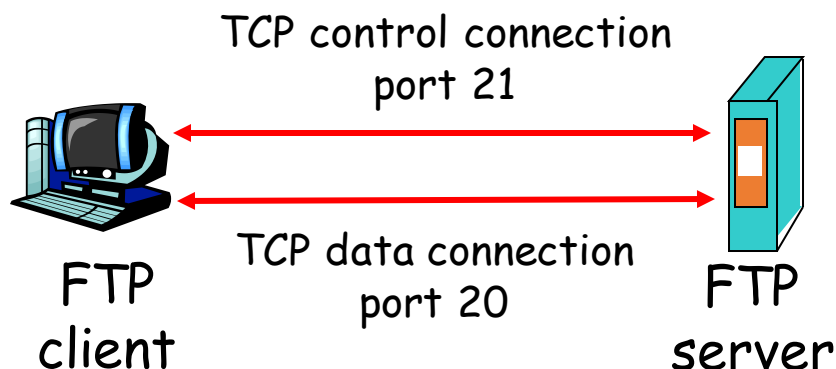
Domain Name System (DNS)

Peer-to-Peer (P2P) Applications



FTP

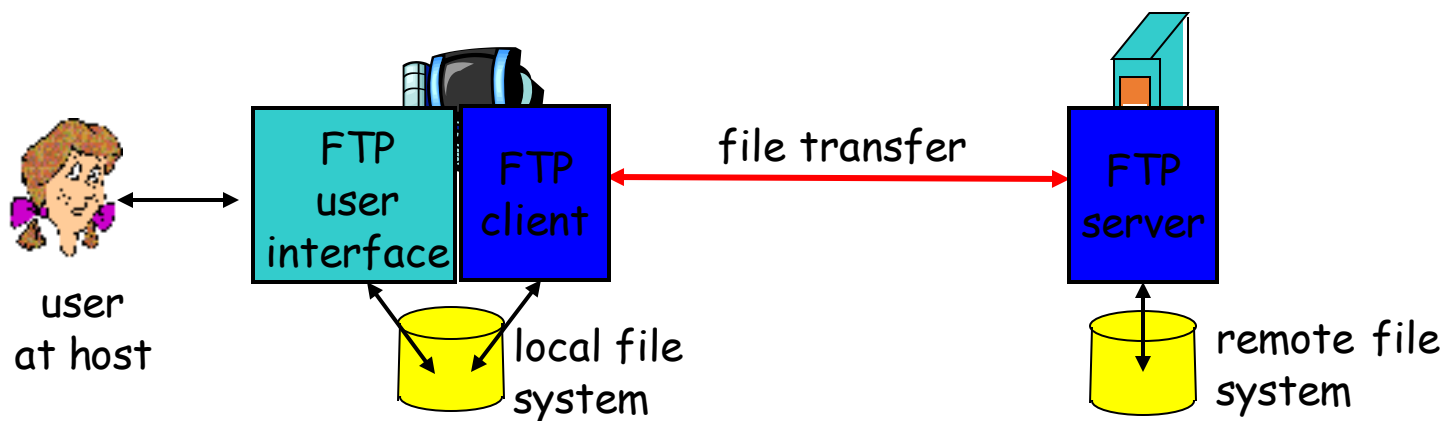
- Commonly used application layer protocol
- Allows for the transfer of files between clients/servers.
- Requires 2 connections to the server
 - ⊙ Commands – uses TCP port 21
 - ⊙ Actual data – uses TCP port 20





FTP: the file transfer protocol

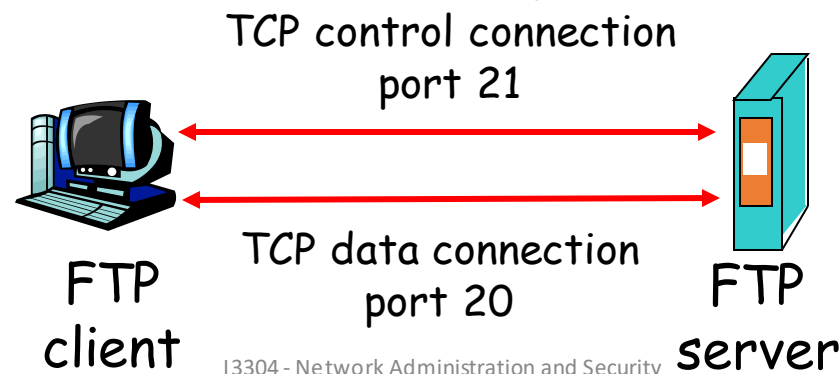
- Transfer file to/from remote host
- Client/server model
 - ◉ *Client*: side that initiates transfer (either to/from remote)
 - ◉ *Server*: remote host
- FTP: RFC 959
- FTP server: **port 21** (control)





FTP: two connections, control & data

- FTP client contacts FTP server at port 21, specifying TCP as transport protocol
- Client obtains authorization over control connection
- Client browses remote directory by sending commands over control connection.
- When server receives file transfer command, server opens 2nd TCP connection (for file) to client
- After transferring one file, server closes data connection.
- Server opens another TCP data connection to transfer another file.
- Control connection: “out of band”
- FTP server maintains “state”: current directory, earlier authentication





FTP: commands - responses

Sample commands:

- sent as ASCII text over control channel
- **USER** username
- **PASS** password
- **LIST** return list of file in current directory
- **RETR filename** retrieves (gets) file
- **STOR filename** stores (puts) file onto remote host

Sample return codes

- status code and phrase (as in HTTP)
- 331 Username OK, password required
- 125 data connection already open; transfer starting
- 425 Can't open data connection
- 452 Error writing file



Introduction

Web and HTTP

File Transfer Protocol (FTP)

Telnet

Mail protocols

Domain Name System (DNS)

Peer-to-Peer (P2P) Applications



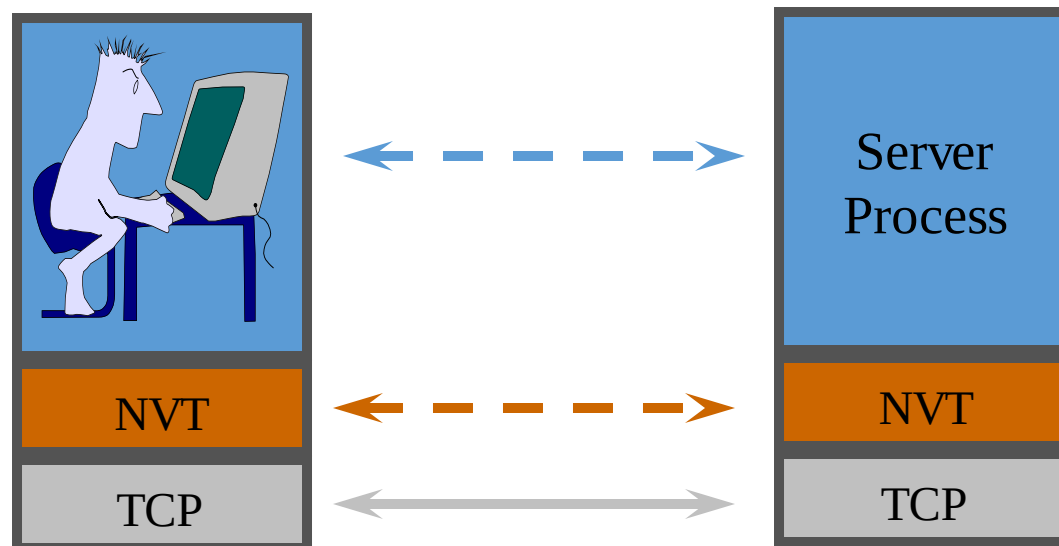
TELNET vs. telnet

- TELNET is a **protocol** that provides “a general, bi-directional, eight-bit byte oriented communications facility”.
- `telnet` is a program that supports the TELNET protocol over TCP.
- Many application protocols are built upon the TELNET protocol.



The TELNET Protocol [RFC 854]

- TCP connection
- data and control over the same connection.
- Network Virtual Terminal (NVT)
 - ◉ Intermediate representation of a generic terminal.
 - ◉ Provides a standard language for communication of terminal control functions.





Playing with TELNET

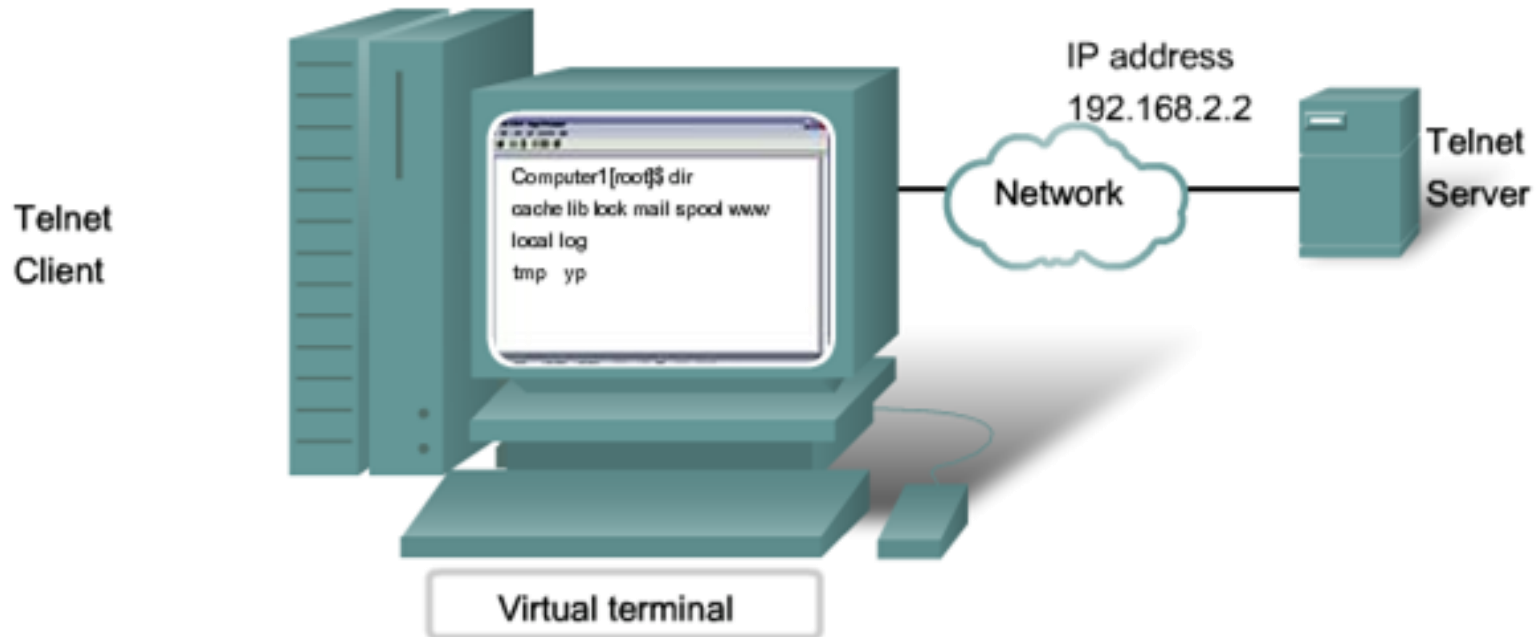
- You can use the `telnet` program to play with the TELNET protocol.
- `telnet` is a generic TCP client.
 - ⦿ Sends whatever you type to the TCP socket.
 - ⦿ Prints whatever comes back through the TCP socket
 - ⦿ Useful for testing TCP servers (ASCII-based protocols).

Telnet



- Developed in the early 1970's – among the oldest of the application layer protocols and services in the TCP/IP protocol suite.
- Allows users to emulate text-based terminal devices over the network using software.
- A connection is known as a **'virtual terminal (vty)' session**.
- Can be run from the command prompt on a PC.
- You can use the device as if you were sitting there with all the rights and priorities that your username will offer you.
- **Disadvantages: Doesn't support encryption** like SSH.
 - ⊙ All data is transferred as plain text.
 - ⊙ It can be easily intercepted and understood.
- **If security is a concern, you should use Secure Shell (SSH) protocol.**
 - ⊙ Provides for remote logins with stronger authentication than telnet.
 - ⊙ Network Professionals should always use SSH whenever possible.

Telnet



Telnet provides a way to use a computer, connected via the network, to access a network device as if the keyboard and monitor were directly connected to the device.

What should I know about Telnet (by ChatGPT)



- **Purpose:** Telnet was originally designed as a way for users to remotely access and manage mainframe computers.
 - ⊙ Today, Telnet is mostly used to access and manage network devices, such as routers, switches, and firewalls.
- **Communication:** Telnet uses a *client-server model*
 - ⊙ a Telnet client establishes a connection to a Telnet server running on a network device.
 - ⊙ Once the connection is established, users can enter commands into the Telnet client, which are then transmitted to the Telnet server for execution.
- **Data Transmission:** Telnet transmits data as plain text, including both commands and the output of those commands.
 - ⊙ This makes Telnet vulnerable to eavesdropping and tampering, as data transmitted over Telnet can be easily intercepted and read by an attacker.

What should I know about Telnet (by ChatGPT)



- **Security:** Telnet is not secure and should not be used for sensitive data transmissions.
 - ⦿ All data transmitted over Telnet is transmitted in plain text, which can be easily intercepted and read by an attacker.
 - ⦿ Alternatives to Telnet, such as Secure Shell (SSH), should be used for secure remote access and management of network devices.
- **Limitations:** Telnet has several limitations, including *limited support for data encryption, limited support for terminal emulation, and a lack of authentication mechanisms*.
- Overall, **Telnet is a legacy protocol** that has been largely replaced by more secure alternatives.
 - ⦿ It is still used in some cases for basic remote access and management of network devices, but **users should be aware of its security limitations and should use secure alternatives whenever possible**.



Introduction

Web and HTTP

File Transfer Protocol (FTP)

Telnet

Mail protocols

Domain Name System (DNS)

Peer-to-Peer (P2P) Applications

E-mail

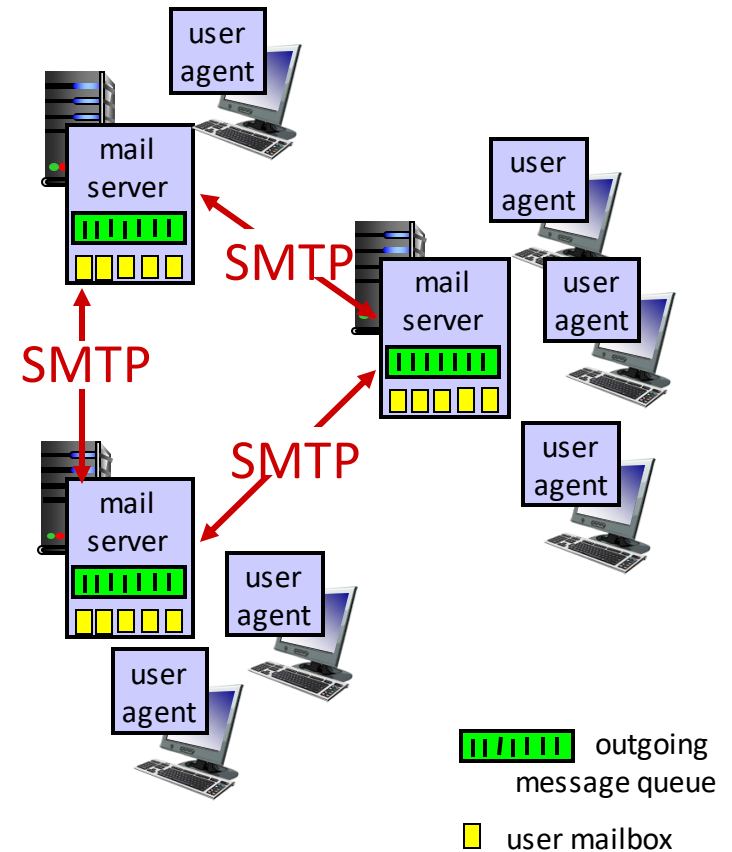


Three major components:

- User Agents
- Mail Servers
- Simple Mail Transfer Protocol: SMTP

User Agent

- a.k.a. “mail reader”
- Composing, editing, reading mail messages
- e.g., Outlook, iPhone mail client
- Outgoing, Incoming messages stored on server



E-mail: mail servers

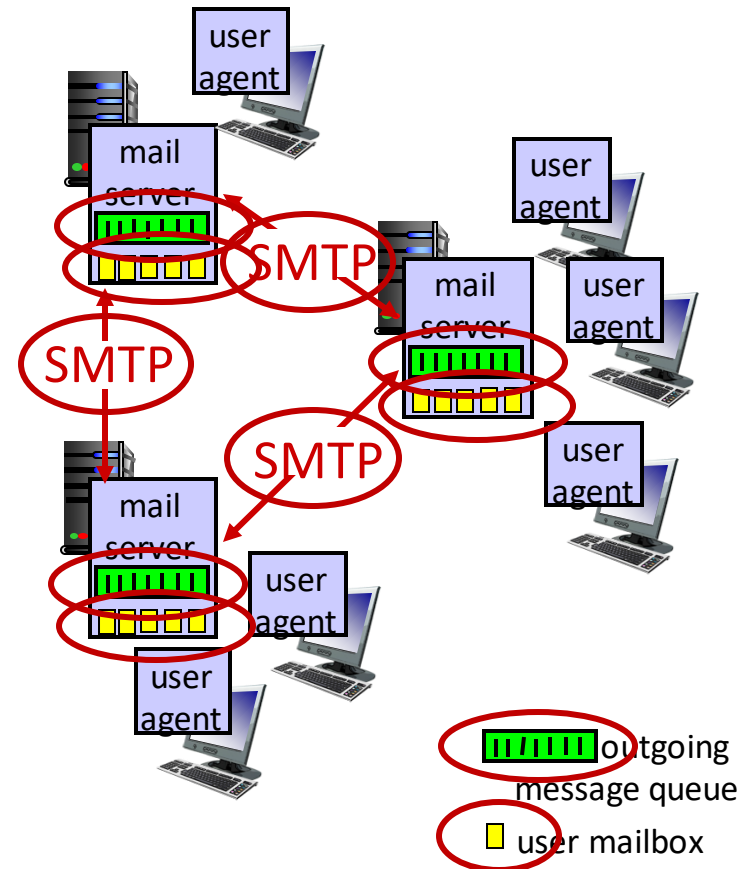


mail servers:

- *mailbox* contains incoming messages for user
- *message queue* of outgoing (to be sent) mail messages

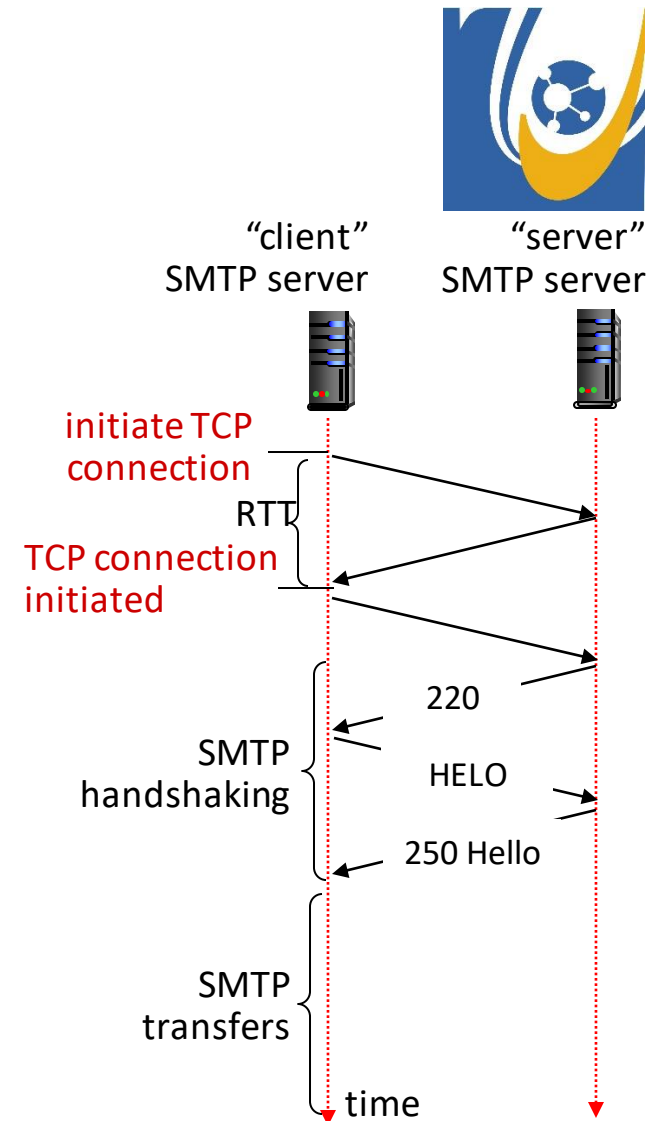
SMTP protocol between mail servers to send email messages

- *client*: sending mail server
- “*server*”: receiving mail server



SMTP RFC (5321)

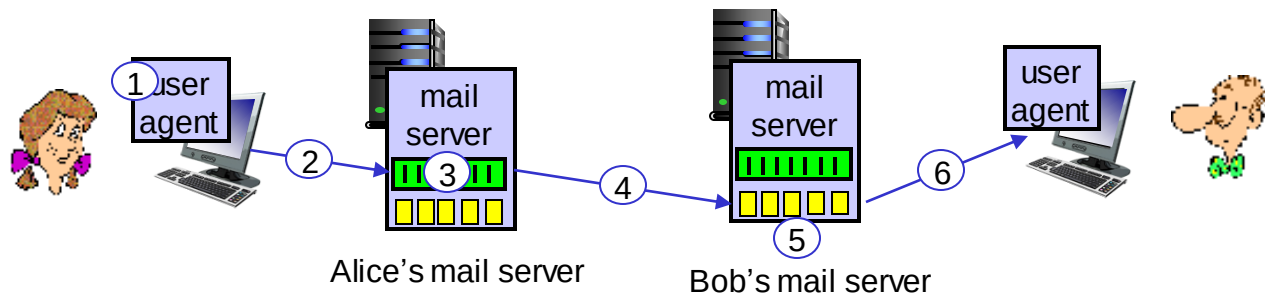
- Uses TCP to reliably transfer email message from client (mail server initiating connection) to server, port 25
 - Direct transfer: sending server (acting like client) to receiving server
- Three phases of transfer
 - ◉SMTP handshaking (greeting)
 - ◉SMTP transfer of messages
 - ◉SMTP closure
- Command/Response interaction (like HTTP)
 - ◉Commands: ASCII text
 - ◉Response: status code and phrase



Scenario: Alice sends e-mail to Bob



1. Alice uses UA to compose e-mail message “to” bob@some school.edu
2. Alice’s UA sends message to her mail server using SMTP; message placed in message queue
3. client side of SMTP at mail server opens TCP connection with Bob’s mail server
4. SMTP client sends Alice’s message over the TCP connection
5. Bob’s mail server places the message in Bob’s mailbox
6. Bob invokes his user agent to read message





SMTP vs. HTTP: Observations

HTTP

- client pull
- ASCII command/response interaction, status codes
- each object encapsulated in its own response message

SMTP

- client push
- ASCII command/response interaction, status codes
- multiple objects sent in multipart message
- uses persistent connections
- requires message (header & body) to be in 7-bit ASCII
- SMTP server uses CRLF .CRLF to determine end of message

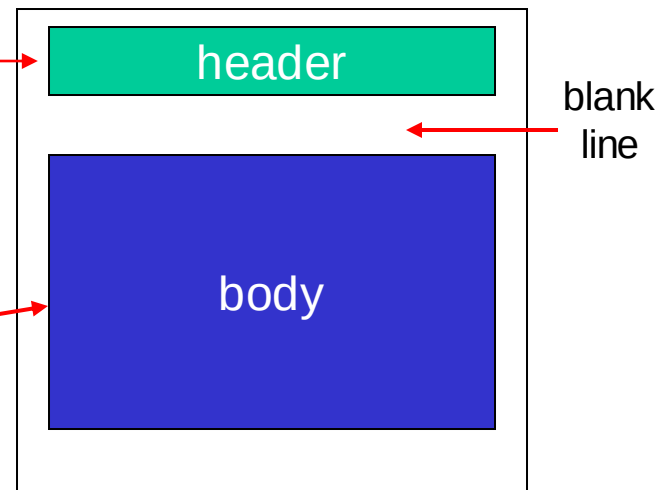


Mail message format

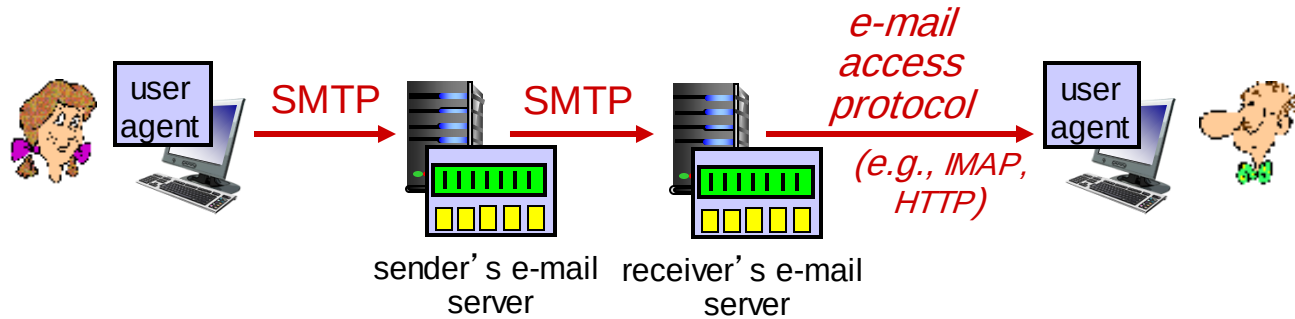
SMTP: protocol for exchanging e-mail messages, defined in RFC 5321 (like RFC 7231 defines HTTP)

RFC 2822 defines *syntax* for e-mail message itself (like HTML defines syntax for web documents)

- header lines, e.g.,
 - To:
 - From:
 - Subject:these lines, within the body of the email message area different from SMTP MAIL FROM:, RCPT TO: commands!
- Body: the “message” , ASCII characters only



Retrieving email: mail access protocols



- **SMTP:** delivery/storage of e-mail messages to receiver's server
- **Mail access protocol:** retrieval from server
 - **IMAP:** Internet Mail Access Protocol [RFC 3501]: messages stored on server, IMAP provides retrieval, deletion, folders of stored messages on server
- **HTTP:** Gmail, Hotmail, Yahoo!Mail, etc. provides web-based interface on top of SMTP (to send), IMAP (or POP) to retrieve e-mail messages

Mail Access Protocol: Post Office Protocol



- Used to retrieve email messages from a remote mail server and store them on a local computer for offline reading.
- There are two main versions of the protocol: POP2 and POP3.
- POP provides a **simple way for users to retrieve their email messages from a mail server and store them on their local computer.**
 - ⦿ The messages can then be read and managed even when the user is offline.

Mail Access Protocol: Post Office Protocol



- **Mail retrieval**

- ⦿ When a user connects to a mail server using POP, the messages are transferred from the server to the user's computer.
- ⦿ The messages are then stored locally and can be read even when the user is offline.

- **Message deletion**

- ⦿ When a user retrieves messages using POP, the messages are typically marked for deletion on the server.
- ⦿ Once the user disconnects from the server, the marked messages are permanently deleted.

- **Security**

- ⦿ POP does not have built-in security features
- ⦿ Typically used with a secure connection (such as SSL or TLS) to protect the privacy of the email messages being retrieved.



Mail Access Protocol: IMAP

- IMAP: Internet Message Access Protocol
- A widely-used Internet standard protocol for managing email messages on a remote mail server.
- Allow users to access their email messages on a mail server **without having to download them to their local computer.**
- Provides a way for users to manage them in a variety of ways.
 - ⊙ This includes reading, moving, copying, and deleting messages, as well as creating and managing email folders.
- **Mail synchronization**
 - ⊙ IMAP allows users to access their email messages from multiple devices and keep them in sync.
 - ⊙ When a user accesses their email messages on one device, any changes they make (such as reading or deleting messages) will be reflected on all other devices that use the same IMAP account.



Mail Access Protocol: IMAP

- **Mail retention**

- ⊙IMAP allows users to **keep their email messages on the server**, which can be useful for backup and synchronization purposes.

- **Advanced features**

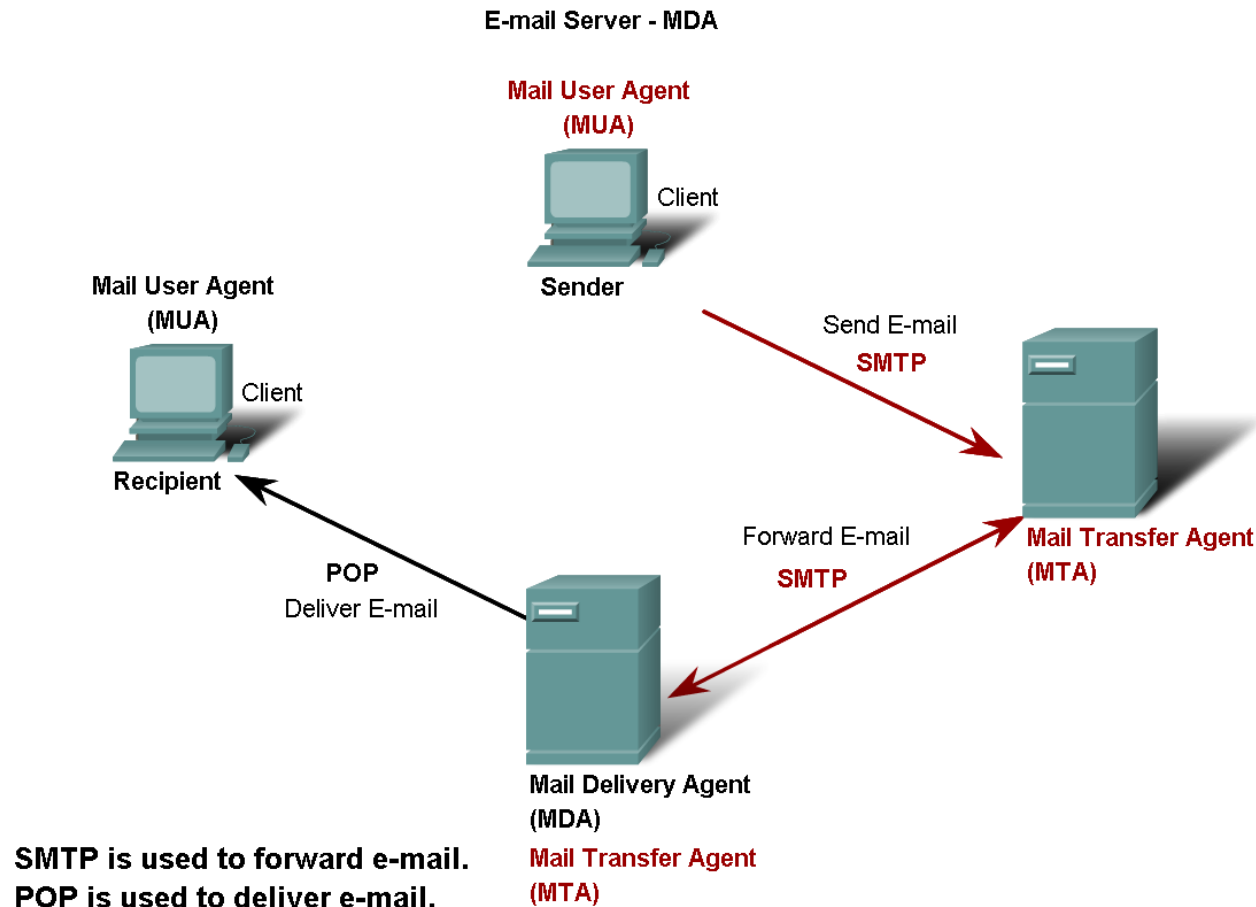
- ⊙IMAP provides advanced features such as server-side searching and message threading, which make it easier for users to manage their email messages and find what they're looking for.

- **Security**

- ⊙IMAP does not have built-in security features

- ⊙Typically used with a secure connection (such as SSL or TLS) to protect the privacy of the email messages being accessed.

E-mail services and SMTP/POP protocols





Introduction

Web and HTTP

File Transfer Protocol (FTP)

Telnet

Mail protocols

Domain Name System (DNS)

Peer-to-Peer (P2P) Applications



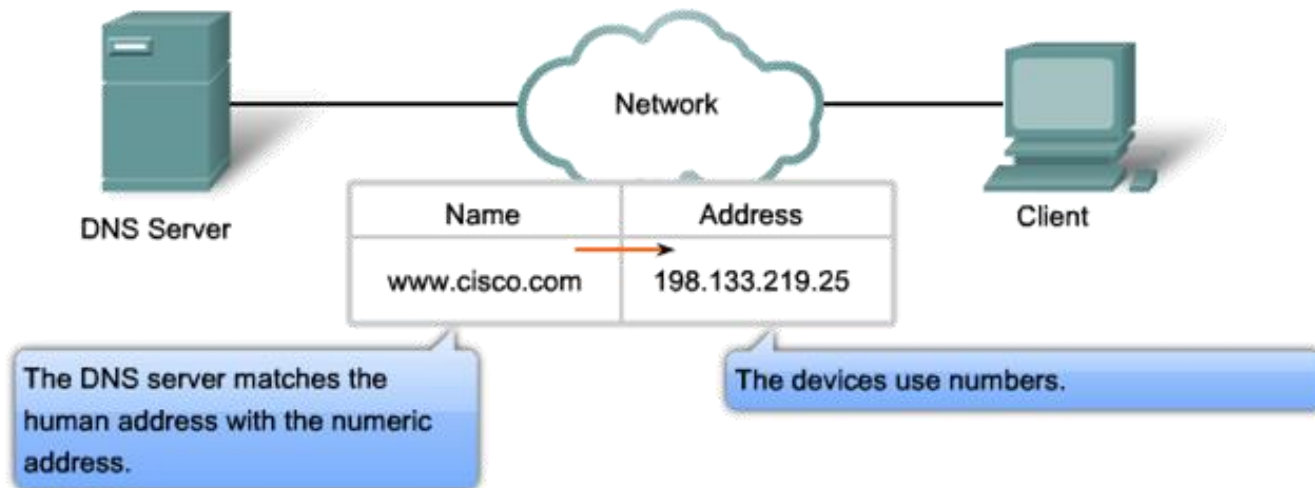
DNS: Domain Name System

- People: many identifiers: SSN, name, passport #
- Internet hosts, routers:
 - ◉ IP address (32/128 bits) - used for addressing datagrams
 - ◉ “name”, e.g., cs.umass.edu - used by humans
- It would be difficult to remember the IP address of every website we like to visit, but we can remember names.
- Question: **how to map between IP address and name, and vice versa ?**



DNS: Domain Name System

- DNS Servers resolve names to IP addresses.
- **Distributed database** implemented in hierarchy of many name servers
- **Application-layer protocol:** hosts, DNS servers communicate to resolve names (address/name translation)
 - ⊙note: core Internet function, implemented as application-layer protocol
 - ⊙complexity at network's "edge"





DNS: services, structure

- **DNS services:**

- ☉hostname-to-IP-address translation

- ☉host aliasing

- canonical, alias names

- ☉mail server aliasing

- ☉load distribution

- replicated Web servers:
many IP addresses
correspond to one name

Q: Why not centralize DNS?

- single point of failure
- traffic volume
- distant centralized database
- Maintenance

Thinking about the DNS

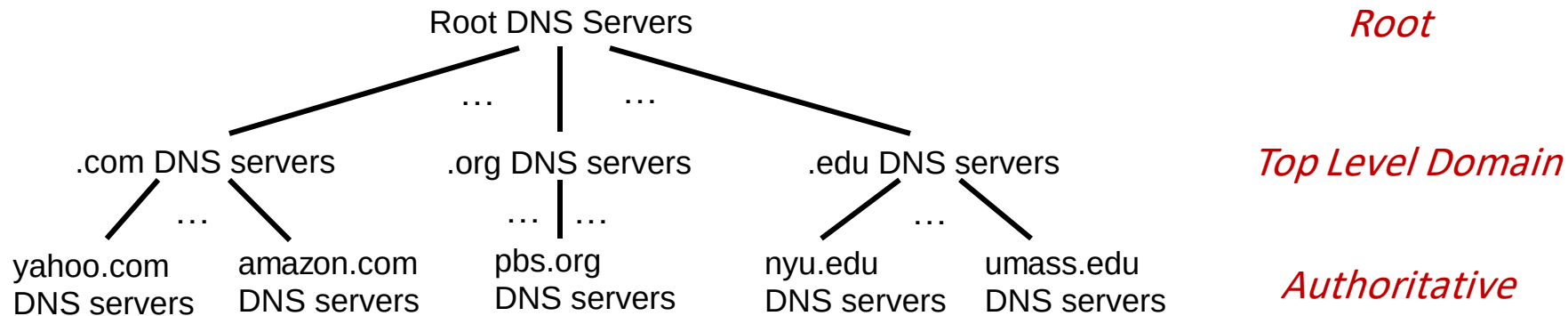


- Humongous distributed database:
 - ⦿ ~ billion records, each simple
- Handles many *trillions* of queries/day:
 - ⦿ *many* more reads than writes
 - ⦿ *performance matters*: almost every Internet transaction interacts with DNS - msec count!
- Organizationally, physically decentralized:
 - ⦿ millions of different organizations responsible for their records
- “Bulletproof”: reliability, security





DNS: a distributed, hierarchical database



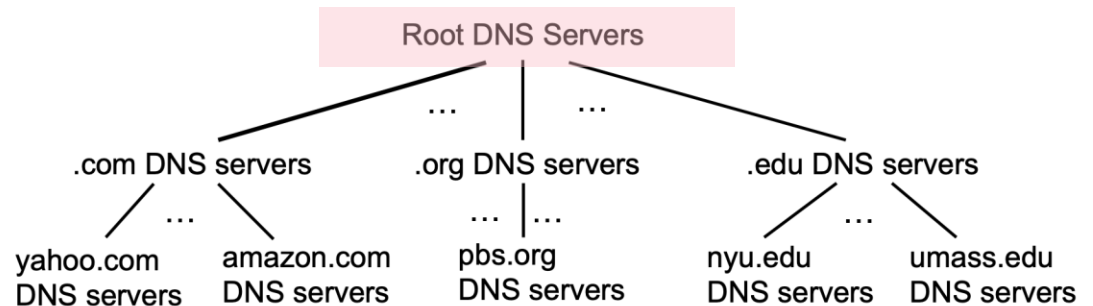
- **Client wants IP address for `www.amazon.com`; 1st approximation:**

- ⦿ client queries root server to find .com DNS server
- ⦿ client queries .com DNS server to get amazon.com DNS server
- ⦿ client queries amazon.com DNS server to get IP address for `www.amazon.com`



DNS: root name servers

- Official, [contact-of-last-resort](#) by name servers that can not resolve name

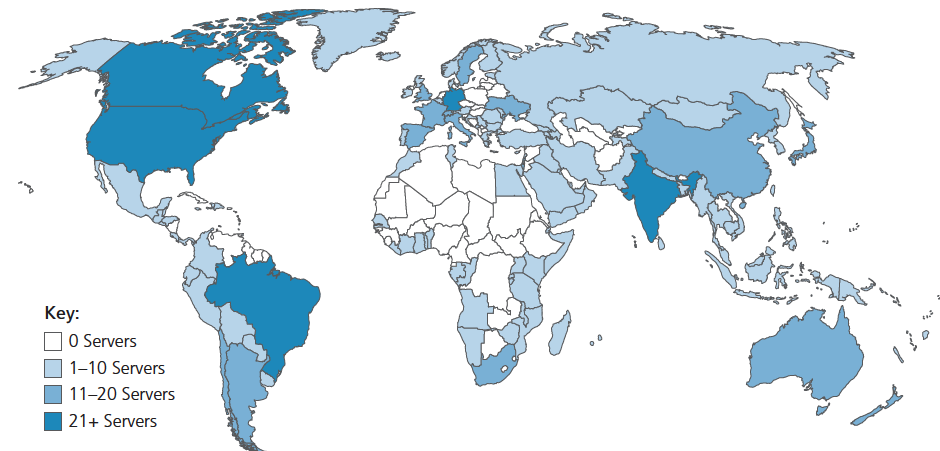




DNS: root name servers

- official, contact-of-last-resort by name servers that can not resolve name
- *incredibly important* Internet function
 - ⊙ Internet couldn't function without it!
 - ⊙ DNSSEC – provides security (authentication, message integrity)
- ICANN (Internet Corporation for Assigned Names and Numbers) manages root DNS domain

13 logical root name “servers”
worldwide each “server” replicated
many times (~200 servers in US)

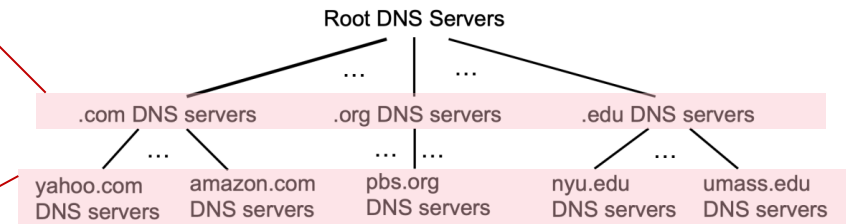


Top-Level Domain, and authoritative servers



Top-Level Domain (TLD) servers:

- responsible for .com, .org, .net, .edu, .aero, .jobs, .museums, and all top-level country domains, e.g.: .cn, .uk, .fr, .ca, .jp
- Network Solutions: authoritative registry for .com, .net TLD
- Educause: .edu TLD



Authoritative DNS servers:

- organization's own DNS server(s), providing authoritative hostname to IP mappings for organization's named hosts
- can be maintained by organization or service provider



Local DNS name servers

- When host makes DNS query, it is sent to its *local*/DNS server
 - ⊙ Local DNS server returns reply, answering:
 - from its local cache of recent name-to-address translation pairs (possibly out of date!)
 - forwarding request into DNS hierarchy for resolution
 - ⊙ Each ISP has local DNS name server; to find yours:
 - MacOS: `% scutil --dns`
 - Windows: `>ipconfig /all`
- Local DNS server doesn't strictly belong to hierarchy

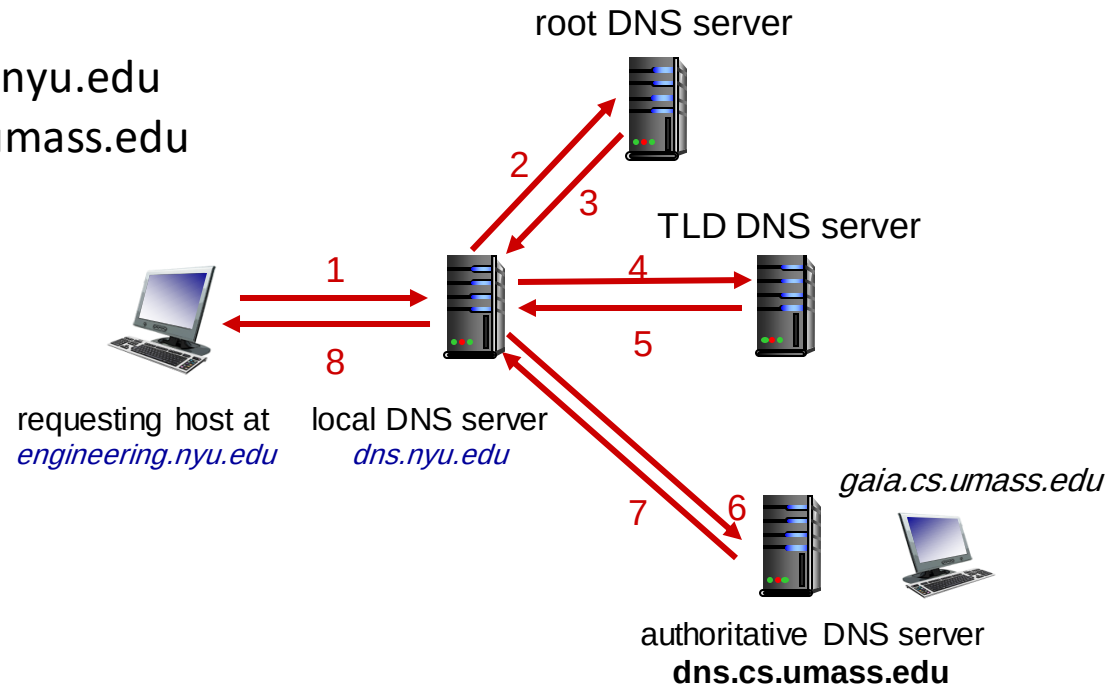
DNS name resolution: iterated query



Example: host at engineering.nyu.edu
wants IP address for gaia.cs.umass.edu

Iterated query:

- contacted server replies with name of server to contact
- “I don’t know this name, but ask this server”



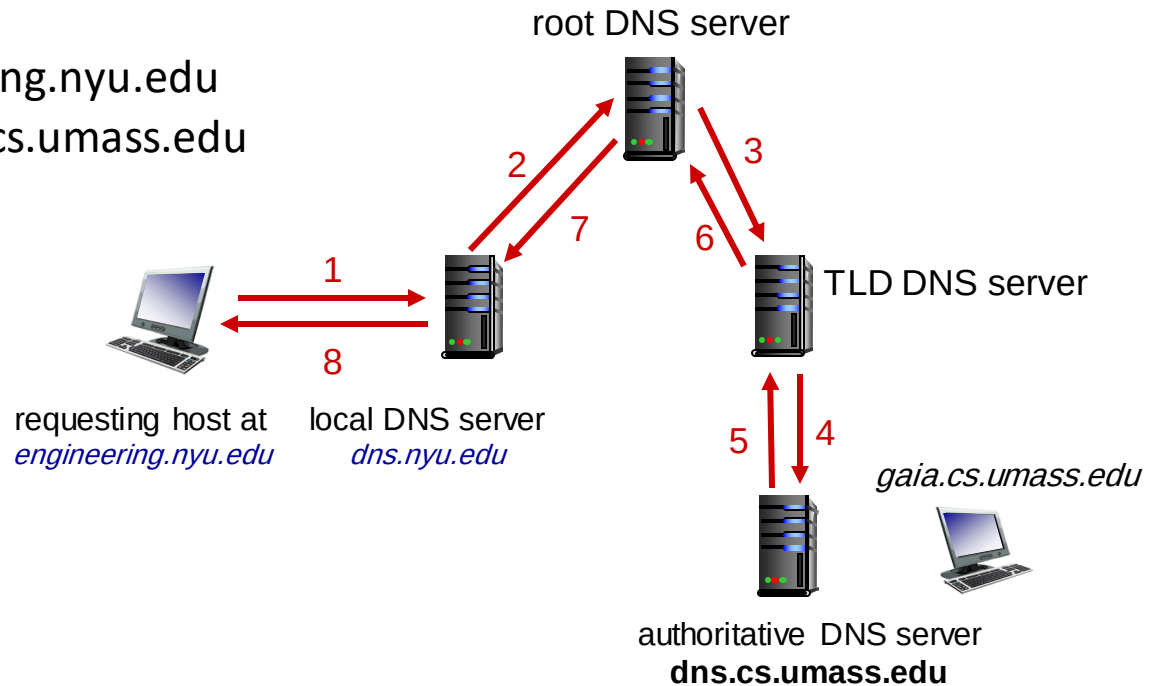
DNS name resolution: recursive query



Example: host at `engineering.nyu.edu` wants IP address for `gaia.cs.umass.edu`

Recursive query:

- puts burden of name resolution on contacted name server
- heavy load at upper levels of hierarchy?





Caching DNS Information

- Once (any) name server learns mapping, it *caches* mapping, and *immediately* returns a cached mapping in response to a query
 - ⦿ caching improves response time
 - ⦿ cache entries timeout (disappear) after some time (TTL)
 - ⦿ TLD servers typically cached in local name servers
- cached entries may be *out-of-date*
 - ⦿ if named host changes IP address, may not be known Internet-wide until all TTLs expire!
 - ⦿ *best-effort name-to-address translation!*

DNS records



DNS: distributed database storing resource records (RR)

RR format: (name, value, type, ttl)

type=A

- name is hostname
- value is IP address

type=NS

- name is domain (e.g., foo.com)
- value is hostname of authoritative name server for this domain

type=CNAME

- name is alias name for some “canonical” (the real) name
- www.ibm.com is really servereast.backup2.ibm.com
- value is canonical name

type=MX

- value is name of SMTP mail server associated with name

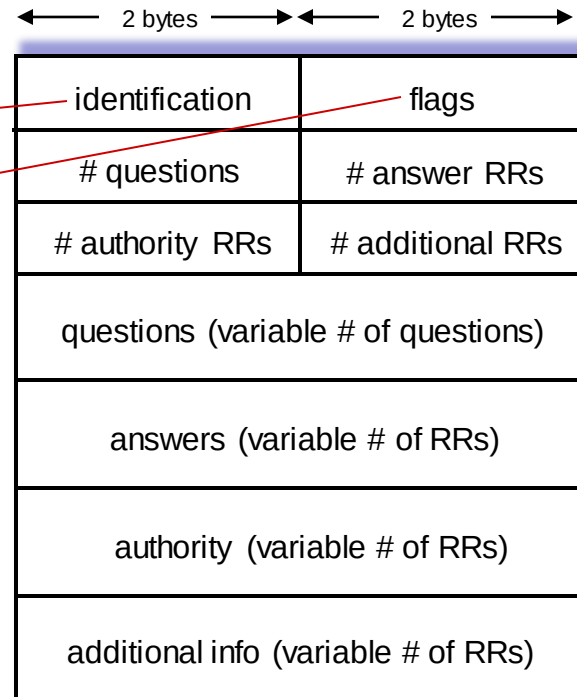
DNS protocol messages



DNS *query* and *reply* messages, both have same *format*:

message header:

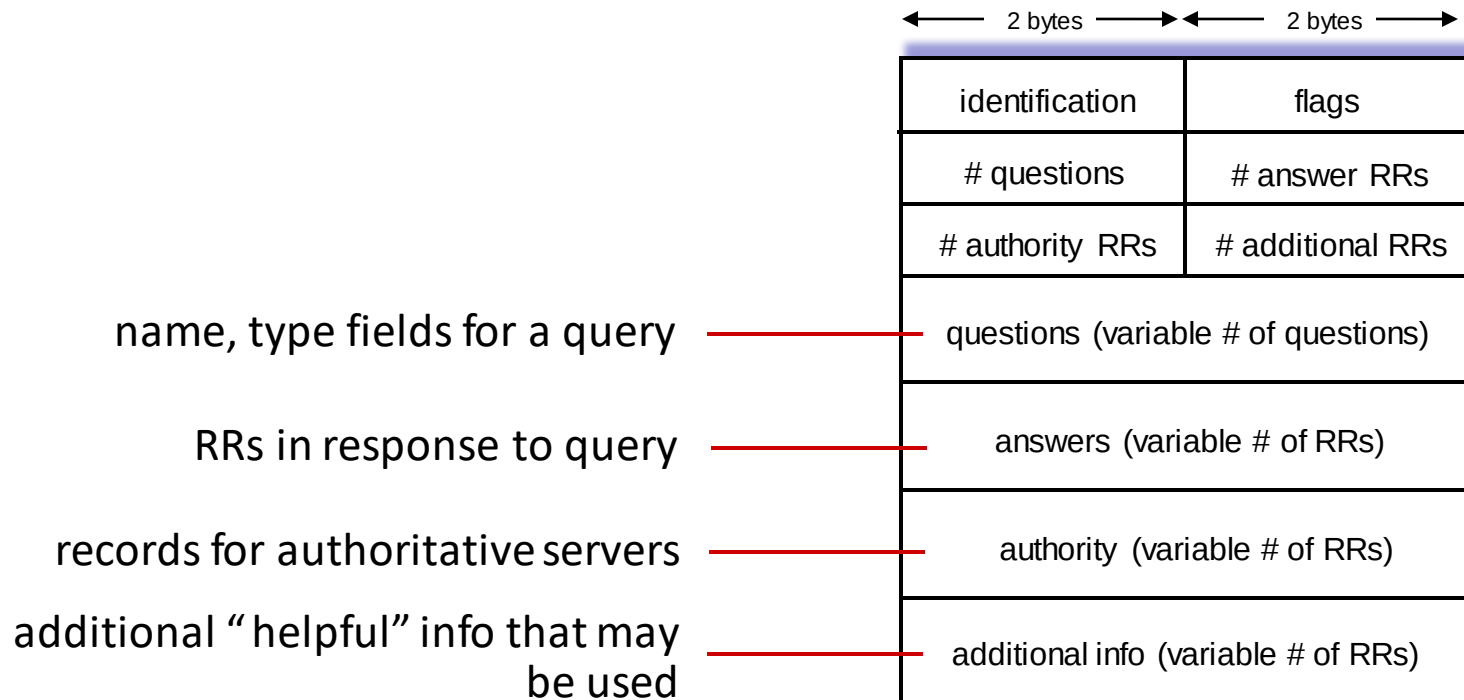
- **identification**: 16 bit # for query, reply to query uses same #
- **flags**:
 - query or reply
 - recursion desired
 - recursion available
 - reply is authoritative



DNS protocol messages



DNS *query* and *reply* messages, both have same *format*:





Getting your info into the DNS

example: new startup “Network Utopia”

- Register name networkutopia.com at *DNS registrar* (e.g., Network Solutions)
 - ⦿ provide names, IP addresses of authoritative name server (primary and secondary)
 - ⦿ registrar inserts NS, A RRs into .com TLD server:
(networkutopia.com, dns1.networkutopia.com, NS)
(dns1.networkutopia.com, 212.212.212.1, A)
- Create **authoritative server** locally with IP address 212.212.212.1
 - ⦿ type A record for www.networkutopia.com
 - ⦿ type MX record for networkutopia.com



DNS security

DDoS attacks

- bombard root servers with traffic
 - ⦿ not successful to date
 - ⦿ traffic filtering
 - ⦿ local DNS servers cache IPs of TLD servers, allowing root server bypass
- bombard TLD servers
 - ⦿ potentially more dangerous

Spoofing attacks

- intercept DNS queries, returning bogus replies
 - DNS cache poisoning
 - RFC 4033: DNSSEC authentication services



Introduction

Web and HTTP

File Transfer Protocol (FTP)

Telnet

Mail protocols

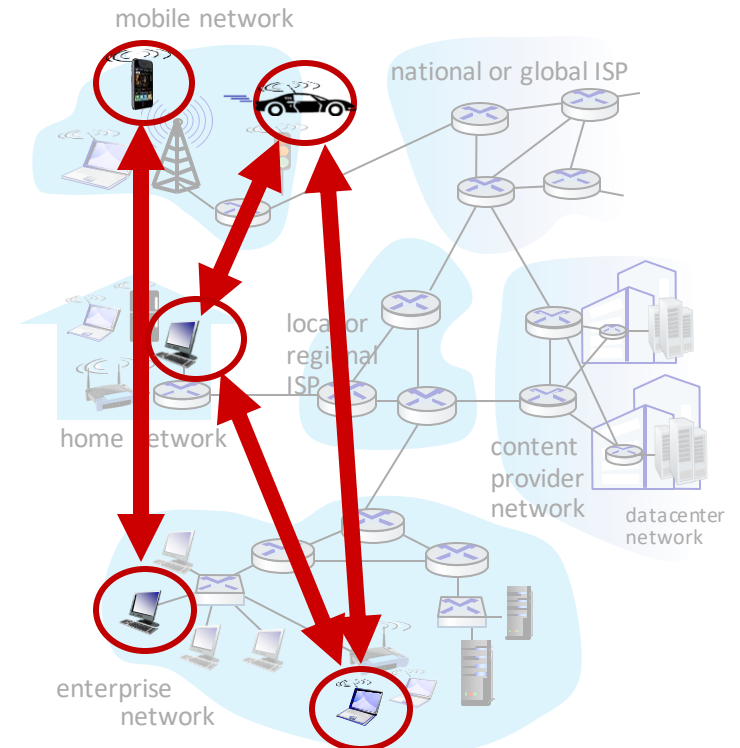
Domain Name System (DNS)

Peer-to-Peer (P2P) Applications



Peer-to-peer (P2P) architecture

- No always-on server
- Arbitrary end systems directly communicate
- Peers request service from other peers, provide service in return to other peers
 - ◉ self scalability : new peers bring new service capacity, and new service demands
- Peers are intermittently connected and change IP addresses
 - ◉ complex management
- E+xamples: P2P file sharing (BitTorrent), streaming (KanKan), VoIP (Skype)

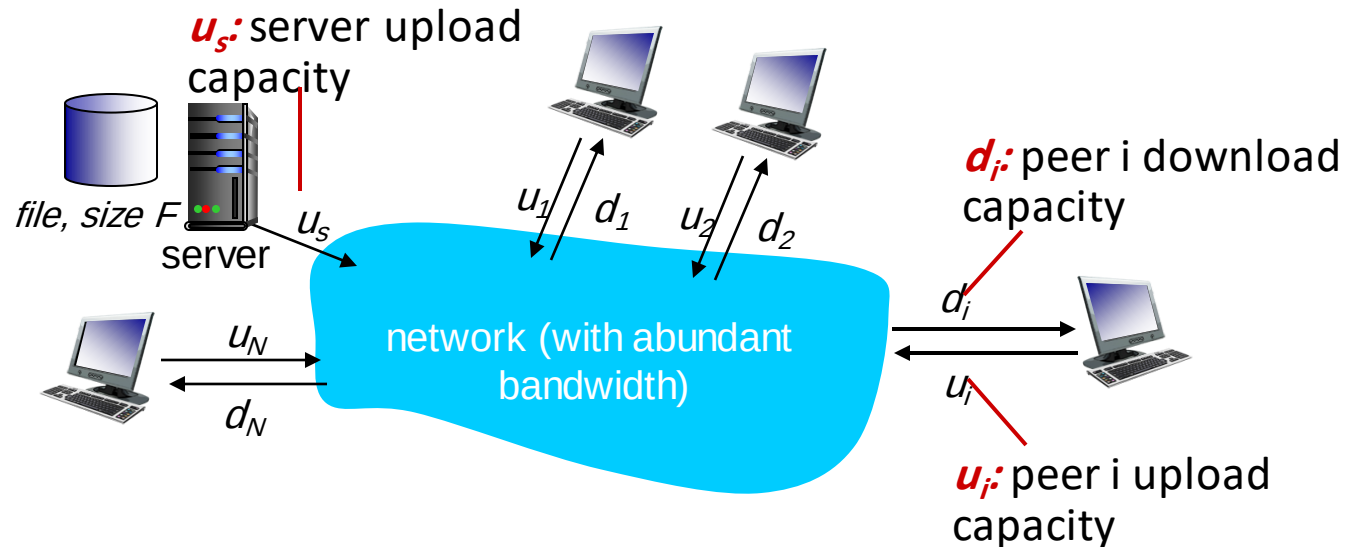


File distribution: client-server vs P2P



Q: how much time to distribute file (size F) from one server to N peers?

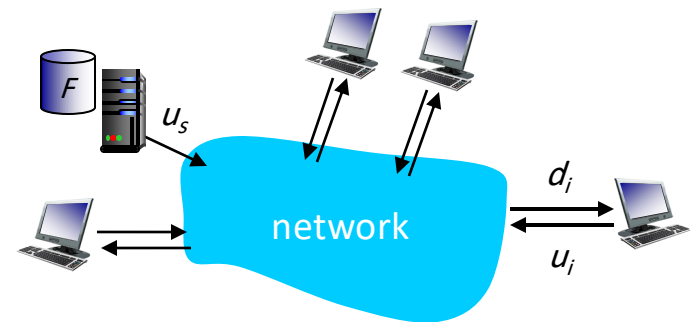
- peer upload/download capacity is limited resource





File distribution time: client-server

- **server transmission:** must sequentially send (upload) N file copies:
 - time to send one copy: F/u_s
 - time to send N copies: NF/u_s
- **client:** each client must download file copy
 - d_{min} = min client download rate
 - min client download time: F/d_{min}



time to distribute F to N clients using client-server approach

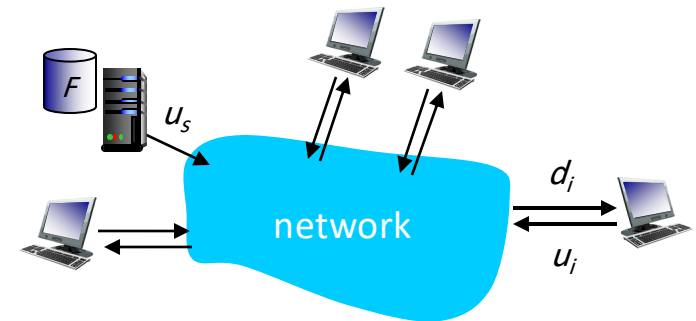
$$D_{c-s} \geq \max\{NF/u_s, F/d_{min}\}$$

increases linearly in N



File distribution time: P2P

- **server transmission:** must upload at least one copy:
 - time to send one copy: F/u_s
- **client:** each client must download file copy
 - min client download time: F/d_{min}
- **clients:** as aggregate must download NF bits
 - max upload rate (limiting max download rate) is $u_s + \sum u_i$



*time to distribute F
to N clients using
P2P approach*

$$D_{P2P} \geq \max\{F/u_s, F/d_{min}, NF/(u_s + \sum u_i)\}$$

increases linearly in N ...
... but so does this, as each peer brings service capacity

Client-server vs. P2P: example



client upload rate = u , $F/u = 1$ hour, $u_s = 10u$, $d_{min} \geq u_s$

